Information Retrieval (AIMLCZG537) - Assignment 2

Complaint & Fraud Ops Workbench

Text mining, classification, clustering and recommendation of bank customer complaints

Group ID: G23

Student Registration Number	Name	Contribution (%)
2025AG05883	Vyas Bhrugu Vipulkumar	100
2025AG05884	Lakshmi Billa	100
2025AG05762	Manas Tripathi	100
2025AG05797	Sadhana Maurya	100
2025AG05747	Stuti Srivastava	100

Dataset: CFPB Consumer Complaint Database - 4,000 complaint narratives, 5 products

Run command: python -m streamlit run app.py

Contents

1. Selected business context
2. Dataset description
3. Implementation approach
4. Preprocessing and feature choices
5. Classification results (Naive Bayes)
6. Clustering results (K-Means)
7. Comparison of classification and clustering
8. Recommendation results
9. Inference and discussion (compulsory questions Q1-Q5)
10. Extra: linear SVM benchmark (beyond the brief)
11. Limitations and improvements for deployment
12. Evidence of execution in the BITS Virtual Lab
Appendix A. Requirement-to-code map
Appendix B. How to run
References

1. Selected business context

We play the text mining team inside a retail bank's complaints and fraud operations unit. Customers send free-text complaints about cards, accounts, payments, debt collectors and credit reports. A large share are about fraud: impersonation scams over Zelle, unauthorised card charges, crypto account take-overs and identity theft on credit reports. The team needs to:

- route every complaint to the right product team without reading it first (classification);
- find recurring problems and scam patterns that nobody has labelled yet (clustering);
- recognise mass-submitted template complaints so they do not distort counts or waste investigator time (duplicate detection);
- pull up similar past complaints when investigating a new case (recommendation).

The Complaint & Fraud Ops Workbench does this end to end in a Streamlit front end. Every algorithm the brief prohibits from libraries (TF-IDF, Naive Bayes, K-Means, Purity, Rand Index) is written by hand in the src/ package, and every step can be seen and changed in the app.

2. Dataset description

The documents are real consumer complaint narratives from the US Consumer Financial Protection Bureau (CFPB) Consumer Complaint Database. Each record has a free-text narrative written by the consumer plus structured fields: product, sub-product, issue, sub-issue, company, state and date. The CFPB stopped publishing narratives in August 2026 [1], so the text comes from an archived full copy of the database on Hugging Face [2] (parquet shards 0059-0065, complaints received May 2020 - Feb 2023, 1.75 million records, 715,568 with a narrative).

2.1 Sampling

Table 1: *Dataset summary*

Property	Value
Products (labels)	Credit card or prepaid card; Checking or savings account; Money transfer, virtual currency or money service; Debt collection; Credit reporting
Filter	narrative of at least 40 words; received 1 Jul 2021 - 31 Dec 2022 (one product taxonomy)
Sample	stratified random sample, 800 complaints per product, random_state = 26 (prepare_data.py)
Documents	4,000
Raw text size	4.92 MB (891,482 words)
Words per complaint	mean 223, median 156, min 39, max 3,753
documents.csv	doc_id (CFPB-<complaint id>), text (narrative only)
metadata.csv	doc_id, title, category, product, sub_product, issue, sub_issue, company, state, date_received, tags, fraud_related, source

Text and metadata are kept in two separate files joined on doc_id. Complaints have no headline, so the title shown in the app is "issue - company". The flag fraud_related marks complaints whose issue or sub-issue mentions fraud, scams, unauthorised use or identity theft; it is metadata for analysis only and is never used as a model feature.

Table 2: *Share of fraud-related complaints per product*

Product	Fraud / scam / ID-theft share
checking_savings	6.1%
credit_card	6.9%
credit_reporting	27.5%
debt_collection	17.5%
money_transfer	54.6%

Money transfer is the most fraud-heavy product (54.6% of complaints), followed by credit reporting (27.5%, mostly identity theft). The most complained-about companies are Equifax, Inc., Jpmorgan Chase & Co., Transunion Intermediate Holdings, Inc., Experian Information Solutions Inc., Bank Of America, National Association.

2.2 Noise in the raw text

Table 3: *Noise profile of the raw complaints (computed on the Data ingestion page)*

Noise type (detected on ingestion)	Documents	% of corpus
XXXX redaction masks	3,417	85.4%
redacted dates (XX/XX/XXXX)	2,129	53.2%
amounts in braces {$...}	1,692	42.3%
encoding artefacts (Â, â€)	0	0.0%
html / markup	0	0.0%
urls / emails	22	0.5%
ALL CAPS words (not XXXX)	1,850	46.2%
numbers / currency	3,235	80.9%
quotes / punctuation heavy	137	3.4%

The dominant noise comes from the CFPB's own privacy masking: names, account numbers and dates are replaced by runs of X ("XXXX", "XX/XX/2021") in 85% of complaints, and money amounts are written as "{$2000.00}". Consumers also write in ALL CAPS, misspell words, paste legal text and run sentences together. All of this is kept in the raw files and handled by switchable preprocessing steps.

2.3 Duplicates and near-duplicates

Table 4: *Duplicates found in the corpus*

Duplicate statistic	Value
Exact duplicate pairs (same normalised text)	97
Near-duplicate pairs (TF-IDF cosine >= 0.85)	3379
5-shingle Jaccard of near pairs	min 0.07, median 0.53
Documents involved in at least one pair	393
Exact pairs filed against different companies / issues	68
Pairs whose documents have different product labels	236
Documents removed when de-duplicating (first copy kept)	316 -> 3,684 left
Removed per product	credit_reporting 233, debt_collection 70, credit_card 9, money_transfer 4

Most duplicates are mass-submitted template letters, a well-known pattern in credit reporting complaints. The same paragraph ("In accordance with the Fair Credit Reporting Act ... 15 U.S.C. 1681 section 602 A. States I have the right to privacy ...") is filed many times against Equifax, Experian and TransUnion, with only the masked parts changing. That is why near pairs can have high cosine but low shingle Jaccard: the boilerplate words carry the TF-IDF weight, while the XXXX runs break the 5-word shingles. 236 near pairs cross two product labels, mostly the same template filed once as credit reporting and once as debt collection. This is genuine label ambiguity in the data.

3. Implementation approach

The system is a Streamlit application (app.py) with six workflow pages that follow the six parts of the brief in order. Each page title uses the brief's wording and a badge shows its step (1-6). The algorithms live in a small package (src/) that is shared by the app and by the experiment scripts that regenerate every number in this report, so screenshots and tables come from the same code.

Table 5: *Workflow pages of the application*

Navigation label	Page title (brief item)	What it provides
Dashboard	1. Dashboard and corpus overview	business context, KPI cards, ops brief (fraud hotspot, template campaigns, top companies), label / length / fraud-share / monthly / company charts, model status
Data ingestion	2. Data upload / ingestion	bundled corpus or upload (CSV, JSON, ZIP of .txt), column mapping, separate text / metadata tables, corpus statistics, noise profile, duplicate detection with side-by-side view
Preprocessing	3. Text preprocessing configuration	configurable pipeline, step-by-step view of one complaint, vocabulary and term-frequency distributions before vs after, Zipf plot, inverted index explorer, document vectors, cosine calculator, similarity heatmap
Text mining hub	4. Text mining and corpus analytics hub	Naive Bayes (metrics, confusion matrix, actual vs predicted), K-Means (initialisation, iteration log, stopping, 2-D scatter, cluster sizes, membership), Purity / Rand Index and comparison, corpus insights, extra SVM benchmark tab
Recommendations	5. Document recommendation panel	Top-K for a selected complaint or a free-text case description, case cards with scores, shared terms and metadata, duplicate filter, explanation and limitations
Performance analytics	6. Performance analytics	stage timings, precision@K, preprocessing comparison, duplicate impact, choice of k

3.1 User interface

- **Top navigation:** the six pages sit in a top navigation bar instead of a sidebar, so the full width is used for tables and charts.
- **Pipeline status bar:** a strip above every page shows which pipeline stages have run in this session (Ingest, Clean, Index + TF-IDF, De-duplicate, Triage, Patterns), with the active corpus and preprocessing setting underneath.
- **Consistent product colours:** each product has one colour and badge that is reused in every chart, table and case card.
- **Light and dark themes:** light by default; a dark theme is defined in .streamlit/config.toml and is switched from the app menu (Settings -> Theme). The Theme button in the status bar explains this.

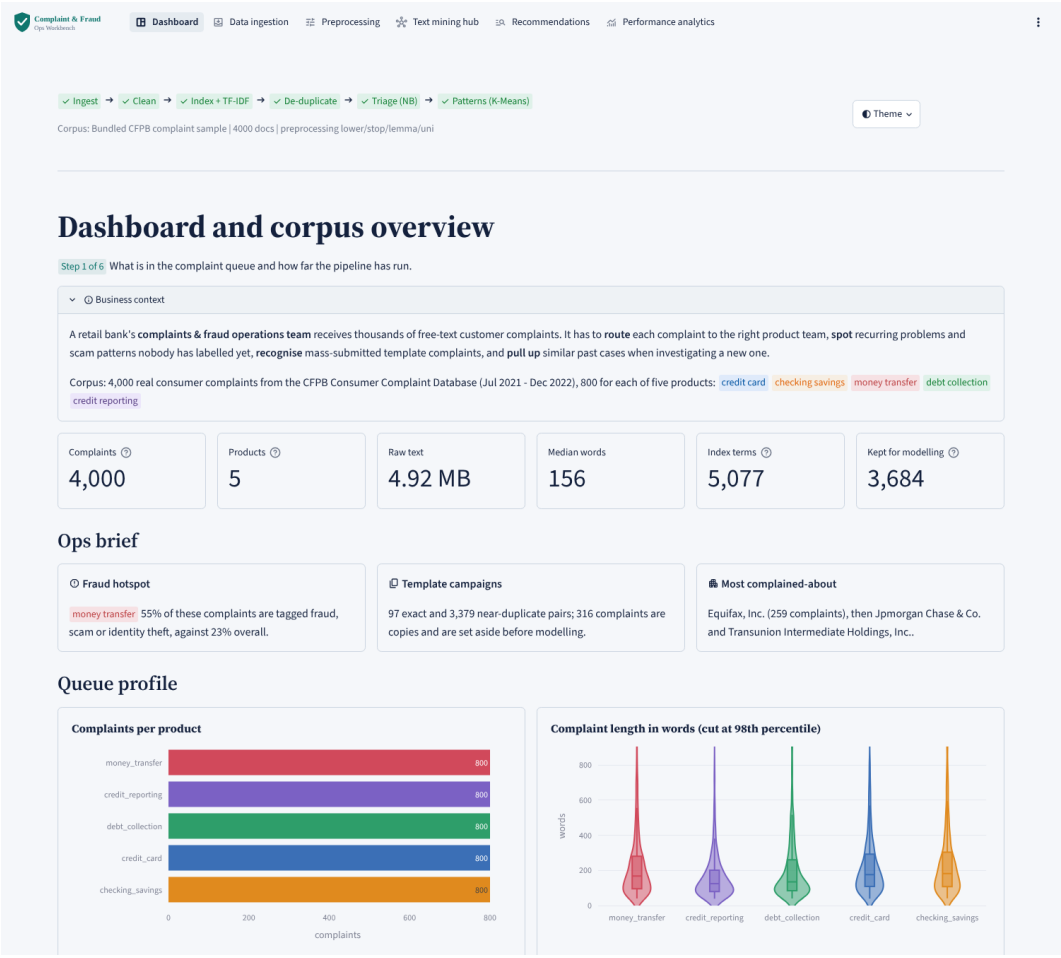


Figure 1: *Dashboard and corpus overview: top navigation, pipeline status bar, business context, KPI cards, ops brief and queue profile*

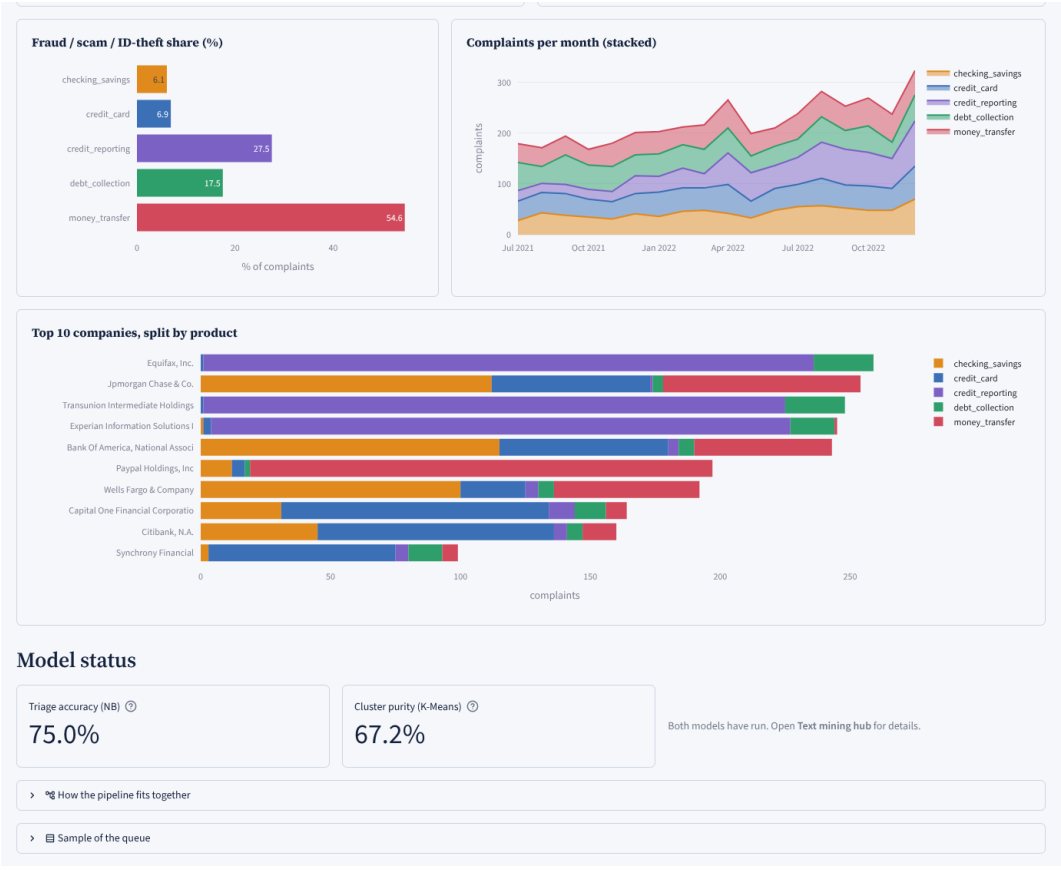


Figure 2: *Dashboard and corpus overview (continued): fraud share, monthly volume, top companies and model status*

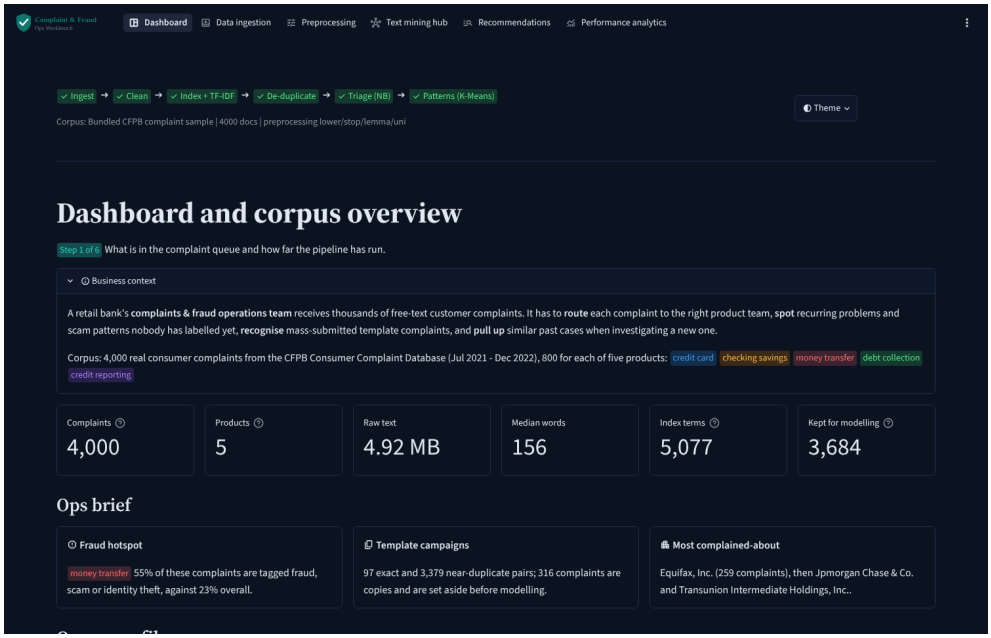


Figure 3: *The same page in the dark theme*

3.2 Architecture

```
ingest (data_io)  ->  clean + tokenise (preprocess)  ->  inverted index + ltc tf-idf (tfidf_index)
                                                                  |
      duplicate detection  <-  cosine similarity matrix  <-+
                                                                  |
   Naive Bayes (naive_bayes)     K-Means (kmeans)     Top-K recommender (recommender)
            \_________________  evaluation (metrics)  _________________/
```

Libraries are used only where the brief allows: NLTK for the stop-word list, Porter stemmer and WordNet lemmatiser; scipy to store the already computed weights in a sparse matrix and for the 2-D projection; numpy for array maths; plotly / matplotlib for charts; scikit-learn's t-SNE only as an optional 2-D projection. No library TF-IDF, Naive Bayes, K-Means, Purity or Rand Index is imported anywhere. The one other use of scikit-learn is the clearly separated SVM benchmark in Section 10, which is not part of the required pipeline.

3.3 Correctness checks and tests

- Rand Index: the contingency-table version is compared with a brute-force O(n^2) loop over all pairs (rand_index_bruteforce in metrics.py); both give identical values.
- TF-IDF: each stored weight equals (1+log10 tf) x log10(N/df) divided by the document's L2 norm, and every document vector has length 1.0000 (shown on the Preprocessing page).
- Cosine: the term-at-a-time score through the inverted index equals the matching row of X.X^T (difference below 1e-7).
- Precision, recall and F1 are derived from the same confusion matrix that is plotted.
- 19 unit tests (tests/) cover the hand-written probabilities, refitting, empty documents, training-only TF-IDF, the K-Means objectives and initialisation edge cases, the SVM benchmark, and a run of every app page (python -m unittest discover -s tests).
- Two of the tests reproduce worked examples from Manning et al. [3]: Fig. 16.4 (Purity 12/17 = 0.71, pair counts TP 20 / FP 20 / FN 24 / TN 72, Rand Index 0.68) and Example 13.1 (multinomial NB assigns the test document to "China", with scores 0.0003 vs 0.0001). Our purity, rand_index and MultinomialNB give exactly the book's numbers.
- Two leakage tests: preprocessing a complaint alone gives the same tokens as preprocessing it inside the corpus, and replacing every test document with different text leaves the trained Naive Bayes model (vocabulary, priors, every term weight) unchanged, for both variants and both feature types. A deliberately leaky version that fits TF-IDF on all documents fails this test.

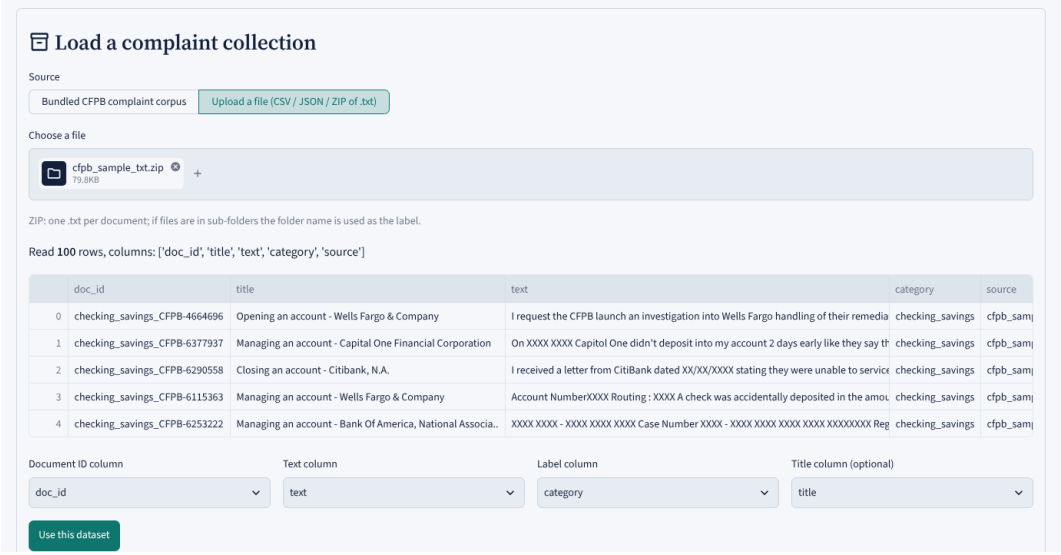


Figure 4: *Data upload / ingestion: a ZIP of .txt files is read and its columns are auto-mapped (folder name = label)*

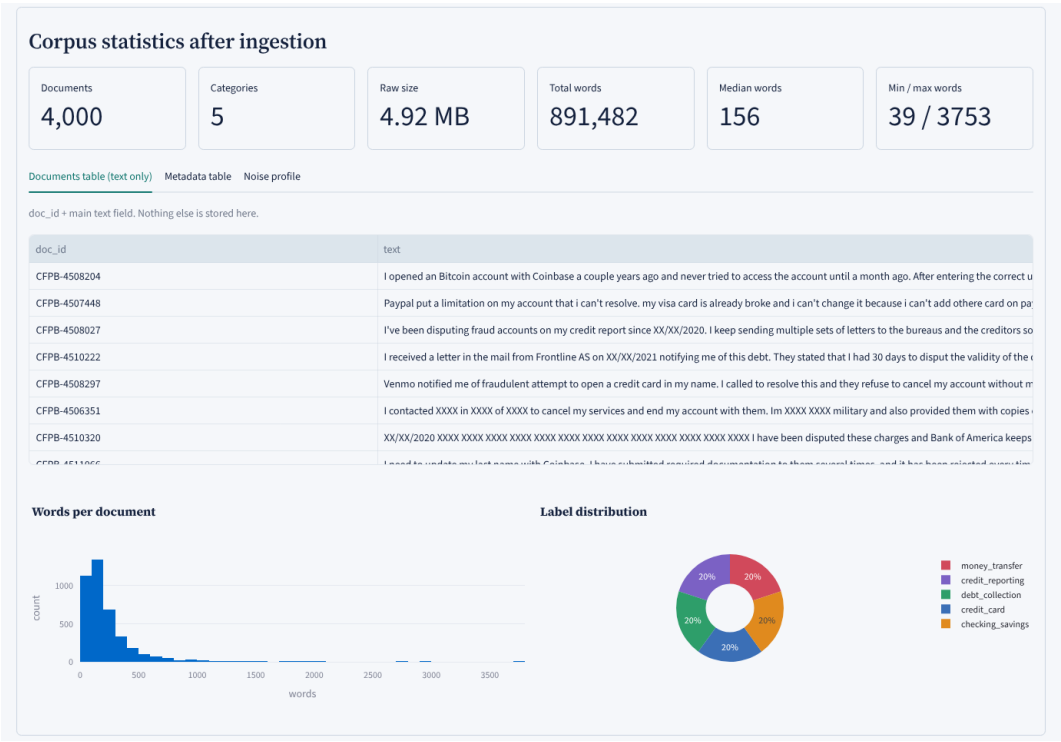
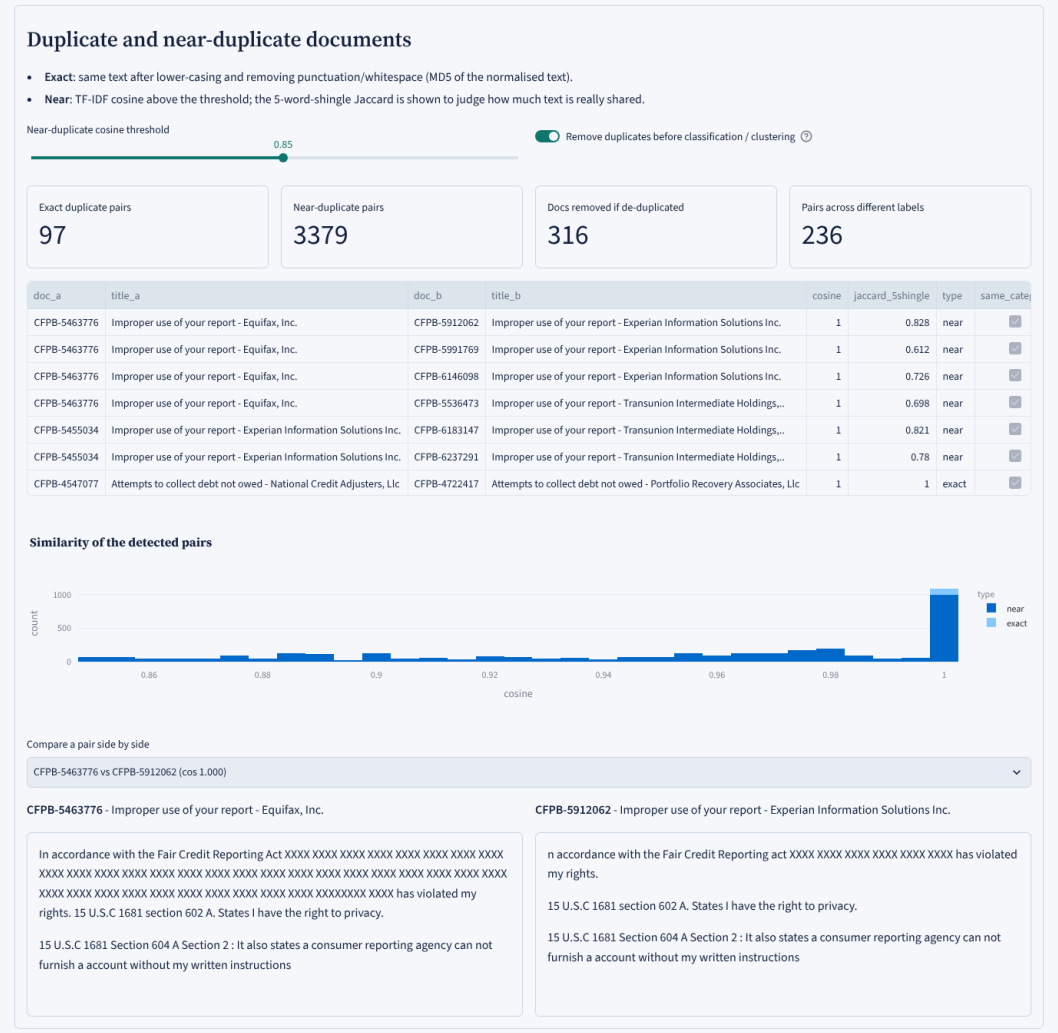


Figure 5: *Corpus statistics after ingestion, with separate document and metadata tables*


Duplicate and near-duplicate documents
Exact: same text after lower-casing and removing punctuation/whitespace (MD5 of the normalised text).
Near: TF-IDF cosine above the threshold; the 5-word-shingle Jaccard is shown to judge how much text is really shared.
Near-duplicate cosine threshold
0.85
Remove duplicates before classification / clustering
Exact duplicate pairs
97
Near-duplicate pairs
3379
Docs removed if de-duplicated
316
Pairs across different labels
236
doc_a title_a doc_b title_b cosine jaccard_5shingle type same_cate
CFPB-5463776 Improper use of your report - Equifax, Inc. CFPB-5912062 Improper use of your report - Experian Information Solutions Inc. 1 0.828 near
CFPB-5463776 Improper use of your report - Equifax, Inc. CFPB-5991769 Improper use of your report - Experian Information Solutions Inc. 1 0.612 near
CFPB-5463776 Improper use of your report - Equifax, Inc. CFPB-6146098 Improper use of your report - Experian Information Solutions Inc. 1 0.726 near
CFPB-5463776 Improper use of your report - Equifax, Inc. CFPB-5536473 Improper use of your report - Transunion Intermediate Holdings,.. 1 0.698 near
CFPB-5455034 Improper use of your report - Experian Information Solutions Inc. CFPB-6183147 Improper use of your report - Transunion Intermediate Holdings,.. 1 0.821 near
CFPB-5455034 Improper use of your report - Experian Information Solutions Inc. CFPB-6237291 Improper use of your report - Transunion Intermediate Holdings,.. 1 0.78 near
CFPB-4547077 Attempts to collect debt not owed - National Credit Adjusters, Llc CFPB-4722417 Attempts to collect debt not owed - Portfolio Recovery Associates, Llc 1 1 exact
Similarity of the detected pairs
count
1000
500
0
0.86
0.88
0.9
0.92
0.94
0.96
0.98
1
cosine
type
near
exact
Compare a pair side by side
CFPB-5463776 vs CFPB-5912062 (cos 1.000)
CFPB-5463776 - Improper use of your report - Equifax, Inc.
In accordance with the Fair Credit Reporting Act XXXX XXXX XXXX

one separately gives identical tokens. Everything that is learnt from data (vocabulary, df filter, idf, Naive Bayes probabilities, the SVM) is fitted on the training split only (Section 4.3 and 5.1). The only steps that see the whole corpus are unsupervised by design and use no labels: duplicate detection, which must compare every pair to find copies before the split, and K-Means and the recommender, which have no train/test split.

4.2 Vocabulary before and after

Table 6: *Vocabulary statistics before and after preprocessing (default configuration)*

Measure	Before (whitespace split)	After preprocessing
Tokens	891,482	379,494
Vocabulary (unique terms)	27,999	8,946
Terms kept in the index (after min_df / max_df)	-	5,077
Type / token ratio	0.0314	0.0236
Hapax legomena (terms seen once)	13,343 (47.7%)	3,433 (38.4%)
Average tokens per document	222.9	94.9
Min / max tokens per document	39 / 3,753	6 / 1,679

Preprocessing removes 57.4% of the tokens and 68.0% of the vocabulary. In the "before" view "XXXX" is the single most frequent token, ahead of "the" and "I". After cleaning, the top terms are the vocabulary of the domain: account, credit, report, bank, call, information, card. The df filter then removes 3,869 more terms. Complaint language is plain and repetitive, so the final vocabulary is small for 4,000 documents (5,077 index terms; the raw text has only about 12,500 distinct lower-case words).

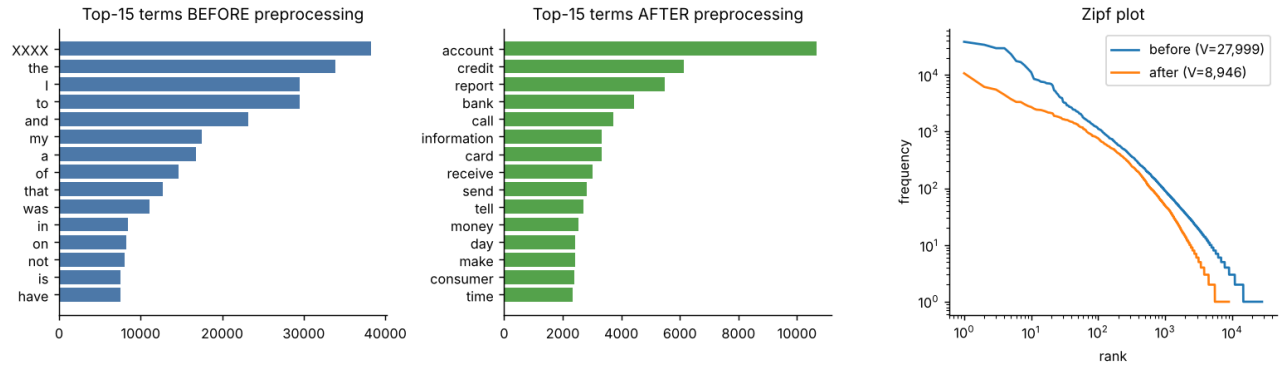


Figure 7: *Top terms before / after preprocessing and Zipf rank-frequency plot*

4.3 Inverted index and TF-IDF (implemented by hand)

src/tfidf_index.py builds the index in one pass. A Counter per document gives the raw posting list term -> [(doc, tf)], and terms outside the df limits are dropped. Each document then gets SMART ltc weights:

$$\mathrm{w(t,d)} = [\ (1 + \log10\ \mathrm{tf(t,d)})\ \mathrm{x}\ \log10(\mathrm{N}\ /\ \mathrm{df(t)})\ ]\ \ /\ \ \mathrm{sqrt(\ sum\ over\ t'\ of\ the\ same\ quantity\ squared\ )}$$

Because every vector is divided by its L2 length (length normalisation), the cosine of two documents is the dot product of their weights. A second posting list term -> [(doc, normalised weight)] lets a query or a seed complaint be scored term-at-a-time (IIR CosineScore), touching only documents that share a term. For K-Means, duplicate detection and precision@K, the computed weights are copied into a sparse matrix X, and the full cosine matrix is X.X^T. For classification, the vocabulary, df filter and idf are fitted on the training split only, and test complaints are transformed with those fitted values (InvertedIndex.transform), so nothing about the test set leaks into the features.

Worked example. The term "zelle" appears in df = 30 of the N = 4,000 complaints, so idf = log10(4000/30) = 2.1249. It is used most in CFPB-5895184 ("Fraud or scam - Early Warning Services, Llc"), tf = 11: tf weight = 1 + log10(11) = 2.0414, raw weight = 2.0414 x 2.1249 = 4.3378. The complaint has 89 index terms and its raw vector length is 14.3663, so the normalised weight is 4.3378 / 14.3663 = 0.3019, the same value stored in the index (0.3019).

Table 7: *Inverted index statistics (default configuration)*

Index statistic	Value
Documents N	4,000
Terms in the index	5,077
Postings (non-zero weights)	233,584
Average posting list length	46.0 documents
Average distinct terms per document	58.4
Sparsity of the document-term matrix	98.85% zeros
Build time (preprocess / index + tf-idf / cosine matrix)	0.86 s / 0.32 s / 0.49 s

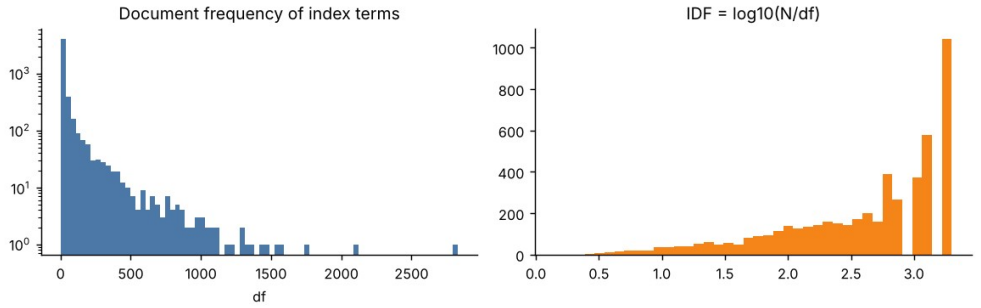


Figure 8: *Document frequency and IDF of the index terms*

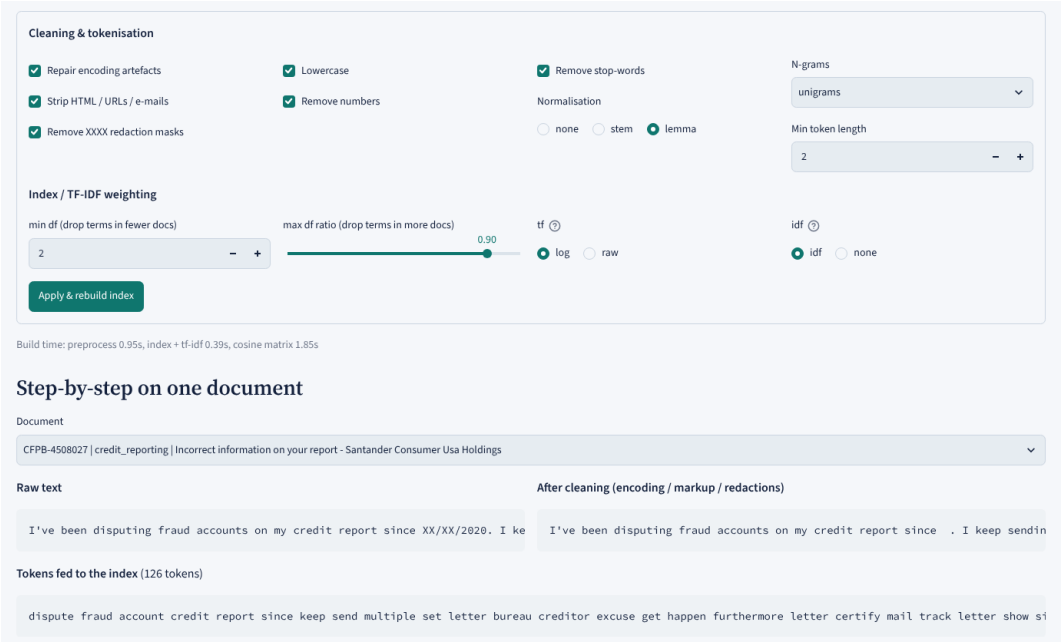


Figure 9: *Text preprocessing configuration: settings and the step-by-step view of one complaint*

Vocabulary before vs after preprocessing

measure	before (whitespace split)	after preprocessing
Tokens	891482	379494
Vocabulary (unique terms)	27999	8946
Terms kept in index (after df filter)	-	5077
Type/token ratio	0.0314	0.0236
Hapax legomena (freq = 1)	13343	3433
Hapax % of vocabulary	47.7	38.4
Avg tokens / doc	222.9	94.9
Min / max tokens per doc	39 / 3753	6 / 1679

Vocabulary reduced by 68.0% ; tokens reduced by 57.4%. The 'before' view keeps case and punctuation glued to words (e.g. 'Zelle', 'zelle,' and 'zelle' are 3 types).

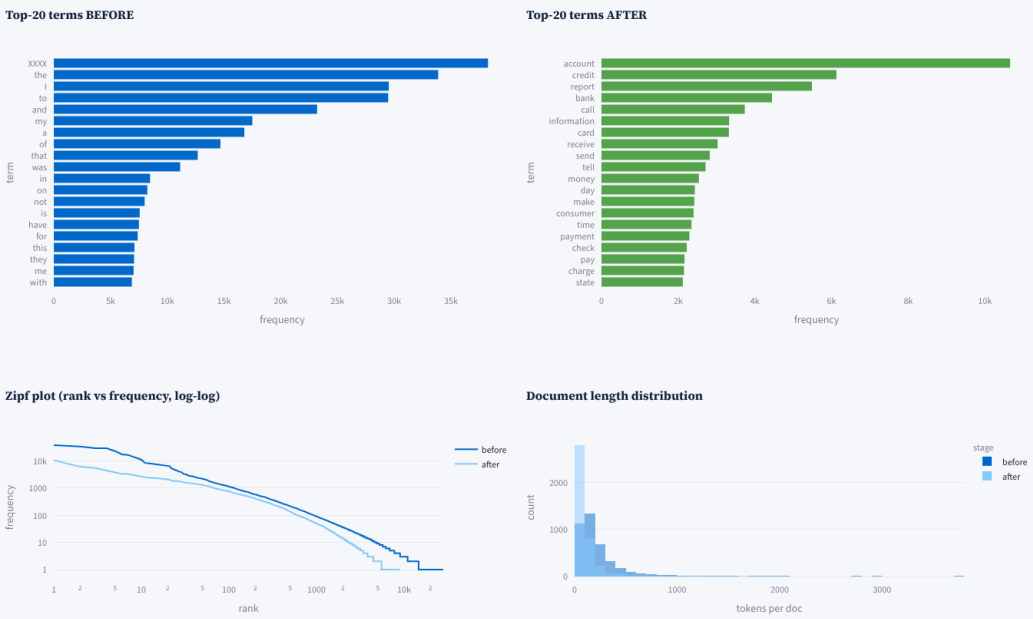


Figure 10: *Text preprocessing configuration: vocabulary before vs after, top terms, Zipf plot and document lengths*

Inverted index & TF-IDF (implemented from scratch)

Terms	Dropped by df filter	Postings	Avg postings / term	Matrix sparsity
5,077	3,869	233,584	46.0	98.85%

$$w_{t,d} = \frac{(1 + \log_{10} tf_{t,d}) \cdot \log_{10}(N/df_t)}{\sqrt{\sum_{t'} [(1 + \log_{10} tf_{t',d}) \cdot \log_{10}(N/df_{t'})]^2}} \qquad \cos(d_1, d_2) = \sum_t w_{t,d_1} w_{t,d_2}$$

Look up a term | Document vector | Cosine of two documents | df / idf distribution

Term (as it appears after preprocessing)

zelle

df = **30**, idf = log10(4000/30) = **2.1249**, collection frequency = **85**

Posting list (top 15 by tf)

tf	doc_id	title	category	normalised weight
11	CFPB-5895184	Fraud or scam - Early Warning Services, Llc	money_transfer	0.3019
9	CFPB-5829877	Fraud or scam - Early Warning Services, Llc	money_transfer	0.3899
5	CFPB-4532493	Managing, opening, or closing your mobile wallet account - Early Warning Services, Llc	money_transfer	0.5502
5	CFPB-5749225	Managing an account - Early Warning Services, Llc	checking_savings	0.3805
5	CFPB-5781791	Fraud or scam - Early Warning Services, Llc	money_transfer	0.3088
5	CFPB-5846253	Fraud or scam - Early Warning Services, Llc	money_transfer	0.5472
4	CFPB-5719918	Fraud or scam - Early Warning Services, Llc	money_transfer	0.2309
3	CFPB-4701439	Fraud or scam - Early Warning Services, Llc	money_transfer	0.1993
3	CFPB-4758703	Fraud or scam - Early Warning Services, Llc	money_transfer	0.3777
3	CFPB-4985205	Other transaction problem - Early Warning Services, Llc	money_transfer	0.2327

Figure 11: *Inverted index explorer: df, idf and the weighted posting list of 'zelle'*

5. Classification results (Naive Bayes)

Key result: Complement Naive Bayes routes 75.0% of the 736 test complaints to the right product (macro-F1 0.751; 78.2% on average over five other splits). Most errors are between products that describe the same event: checking vs money transfer, and debt collection vs credit reporting.

5.1 Method

src/naive_bayes.py implements two Naive Bayes variants from scratch, both on term counts and in log space:

- **Multinomial NB (MNB):** P(t|c) = (count(t,c) + alpha) / (total(c) + alpha |V|) with add-alpha smoothing; a complaint goes to the product with the largest log P(c) + sum x_t log P(t|c) (IIR ch. 13).
- **Complement NB (CNB, default):** each product is scored against the pooled counts of all the other products: w(t,c) = -log((count(t, not c) + alpha) / (total(not c) + alpha |V|)), and the score is sum x_t w(t,c) without a prior. A term that is rare outside a product is evidence for it. With five products, each complement estimate uses four products' worth of text, so the estimates are steadier than the per-class ones of MNB.

The data are split 80/20, stratified by product (seed 42), after de-duplication, so no complaint has a copy on the other side. The variant and alpha were chosen without looking at the test set: three validation splits inside the training part compared MNB and CNB, counts and tf-idf features, and alpha in {0.1, 0.5, 1, 2}.

Table 8: *Best six Naive Bayes settings on the validation splits (mean of 3, training data only)*

Variant	Features	alpha	Validation accuracy	Validation macro-F1
complement	counts	1.0	0.7768	0.7784
complement	counts	2.0	0.7746	0.7763
complement	counts	0.5	0.7740	0.7752
multinomial	counts	0.5	0.7661	0.7675
complement	counts	0.1	0.7633	0.7650
multinomial	counts	0.1	0.7633	0.7644

Complement NB on counts with alpha = 1 had the best validation macro-F1 and is the default in the app; the Naive Bayes tab can switch to MNB or tf-idf features. Counts beat tf-idf weights for both variants, which fits the multinomial model: it expects word counts, and length normalisation removes information it uses.

5.2 Results

Table 9: *Naive Bayes evaluation on the test set*

Complement Naive Bayes	Value
Training / test documents	2,948 / 736
Accuracy (seed-42 split)	0.7500
Macro precision / recall / F1	0.7531 / 0.7524 / 0.7514
Weighted precision / recall / F1	0.7541 / 0.7500 / 0.7508
Accuracy over 5 other splits (seeds 0-4)	0.7821 +/- 0.0098
Training / prediction time	0.15 s / 0.03 s

The seed-42 split happens to be the hardest one: accuracy is 75.0% there and 78.2% on average over five other splits. We report the seed-42 numbers because that is the split behind the confusion matrix and the app screenshots.

Table 10: *Per-class metrics*

Product	Precision	Recall	F1	Support
checking_savings	0.673	0.719	0.695	160
credit_card	0.796	0.766	0.781	158
credit_reporting	0.714	0.796	0.753	113
debt_collection	0.828	0.726	0.774	146
money_transfer	0.755	0.755	0.755	159

Table 11: *Confusion matrix (rows = actual, columns = predicted)*

actual \ predicted	checking_savings	credit_card	credit_reporting	debt_collection	money_transfer
checking_savings	115	12	2	3	28
credit_card	19	121	10	3	5
credit_reporting	1	5	90	15	2
debt_collection	4	8	24	106	4
money_transfer	32	6	0	1	120

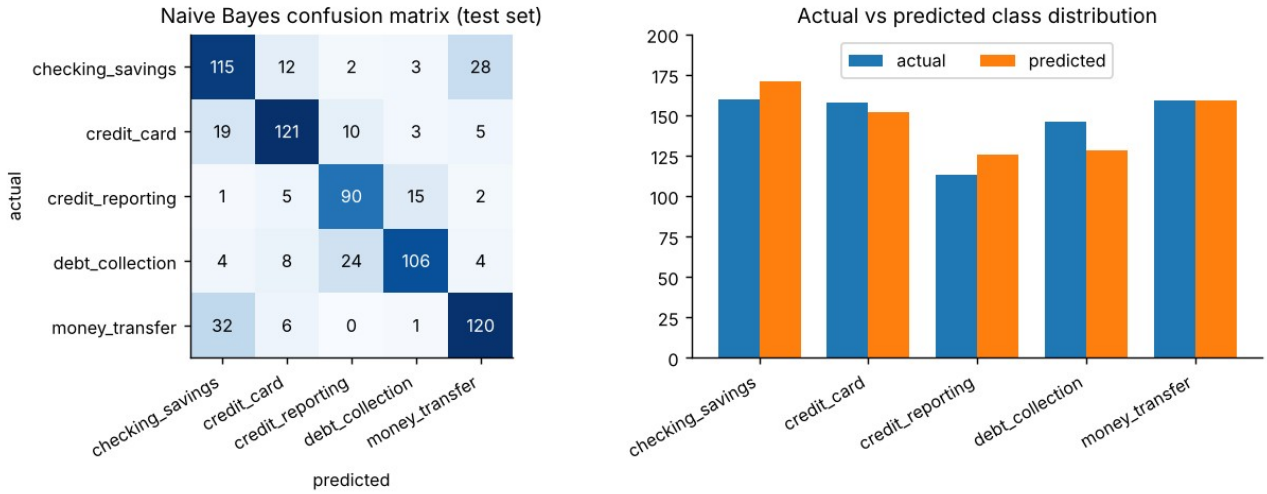


Figure 12: *Naive Bayes confusion matrix and actual vs predicted product distribution*

Table 12: *Predicted labels for ten test complaints, one correct and one wrong per product (all 736 are in results/nb_predictions.csv and on the Naive Bayes tab of the Text mining hub)*

doc_id	Issue - company	Actual	Predicted	Correct
CFPB-4512051	Problem with a purchase shown on your statement - Capit	credit_card	credit_card	yes
CFPB-4512011	Incorrect information on your report - Encore Capital G	credit_reporting	credit_reporting	yes
CFPB-4524623	Took or threatened to take negative or legal action - O	debt_collection	debt_collection	yes
CFPB-4532446	Managing an account - Huntington National Bank, The	checking_savings	checking_savings	yes
CFPB-4536193	Managing, opening, or closing your mobile wallet accoun	money_transfer	money_transfer	yes
CFPB-4516082	Managing an account - Paypal Holdings, Inc	checking_savings	money_transfer	no
CFPB-4519935	Fraud or scam - Chime Financial Inc	money_transfer	checking_savings	no
CFPB-4537279	Attempts to collect debt not owed - Security Credit Ser	debt_collection	credit_reporting	no
CFPB-4539705	Incorrect information on your report - Transunion Inter	credit_reporting	checking_savings	no
CFPB-4638341	Problem with a purchase shown on your statement - Jpmor	credit_card	money_transfer	no

Where the errors are:

- **Checking vs money transfer:** 32 money-transfer complaints were predicted as checking and 28 checking complaints as money transfer. Both are about moving money out of a bank account (Zelle, PayPal, Chime). Which product the consumer chose on the CFPB form is partly arbitrary.
- **Debt collection vs credit reporting:** 24 debt-collection complaints went to credit reporting and 15 the other way. Both complain about a debt on a credit report and often use the same FCRA / FDCPA template text.
- **Credit card -> checking:** 19 card complaints were predicted as checking, mostly debit-card or bank-card disputes where the narrative talks about "my bank" and "my account".

The most indicative terms per product show that the model leans heavily on company names:

Product	Top indicative terms
checking_savings	suntrust, bbva, axos, cd, moneylion, ppp, hsbc, wsfs, iolta, fifththird, mercury, probate
credit_card	reward, tjx, walmart, flight, comenity, amex, kohl, victoria, citicards, costco, gasoline, applicant
credit_reporting	ed, edxxxx, transunion, reinvestigate, navients, chargeoff, equifax, nonaffiliated, ecmc, nv, experian, conviction
debt_collection	ars, glasser, frontline, gym, resurgent, midland, lvnv, hyundai, sequium, ccs, summon, homeowner
money_transfer	binance, gemini, venmo, moneygram, western, cashapp, etoro, wr, skrill, paysafe, eth, usdt

Binance, Gemini and Venmo point to money transfer; TransUnion, Equifax and Experian to credit reporting; Midland, LVNV and Resurgent to debt collection. This is useful in practice, but it means part of the accuracy

comes from "which company" rather than "what happened". A complaint about Wells Fargo is hard to place because Wells Fargo offers every product.

Why the accuracy is lower than on news corpora. Benchmark news collections such as BBC News reach 95%+ with Naive Bayes because sport, politics and technology use almost disjoint vocabularies. Here the labels are the product the consumer picked on the CFPB form, and the same story ("a scammer took money from my account through Zelle") is legitimately filed as checking or as money transfer. The data show this directly: 236 near-identical pairs carry two different product labels, and even a discriminative linear SVM on richer features stops at 79.3% (80.8% on average, Section 10). The ceiling is set by label overlap, not by the implementation.

Confidence. NB scores are over-confident because every word is treated as independent evidence: 63 of the 184 errors have a top score of 0.99 or more. The score still ranks well: complaints scored 0.99 or above are 84.9% correct, while the 23.0% scored below 0.9 are only 55.0% correct. In practice those low-score complaints are the ones to send to a human for routing.

5.3 Multinomial vs Complement NB

Table 13: *Naive Bayes variants on the same splits*

Model (counts, alpha = 1)	Accuracy, seed 42	Macro-F1, seed 42	Accuracy, seeds 0-4
Multinomial NB	0.7296	0.7314	0.7652 +/- 0.0068
Complement NB	0.7500	0.7514	0.7821 +/- 0.0098

CNB is 2.0 points better on the reference split and 1.7 points better on average over the other five. To check whether the reference-split gain is more than chance we ran an exact McNemar test on the 736 test complaints: CNB fixes 36 complaints that MNB got wrong and breaks 21 that MNB got right, p = 0.063. That is not significant at the 5% level on one split. The gain is consistent across the five other splits, so we keep CNB as the default, but the honest summary is a small, consistent improvement rather than a proven one.

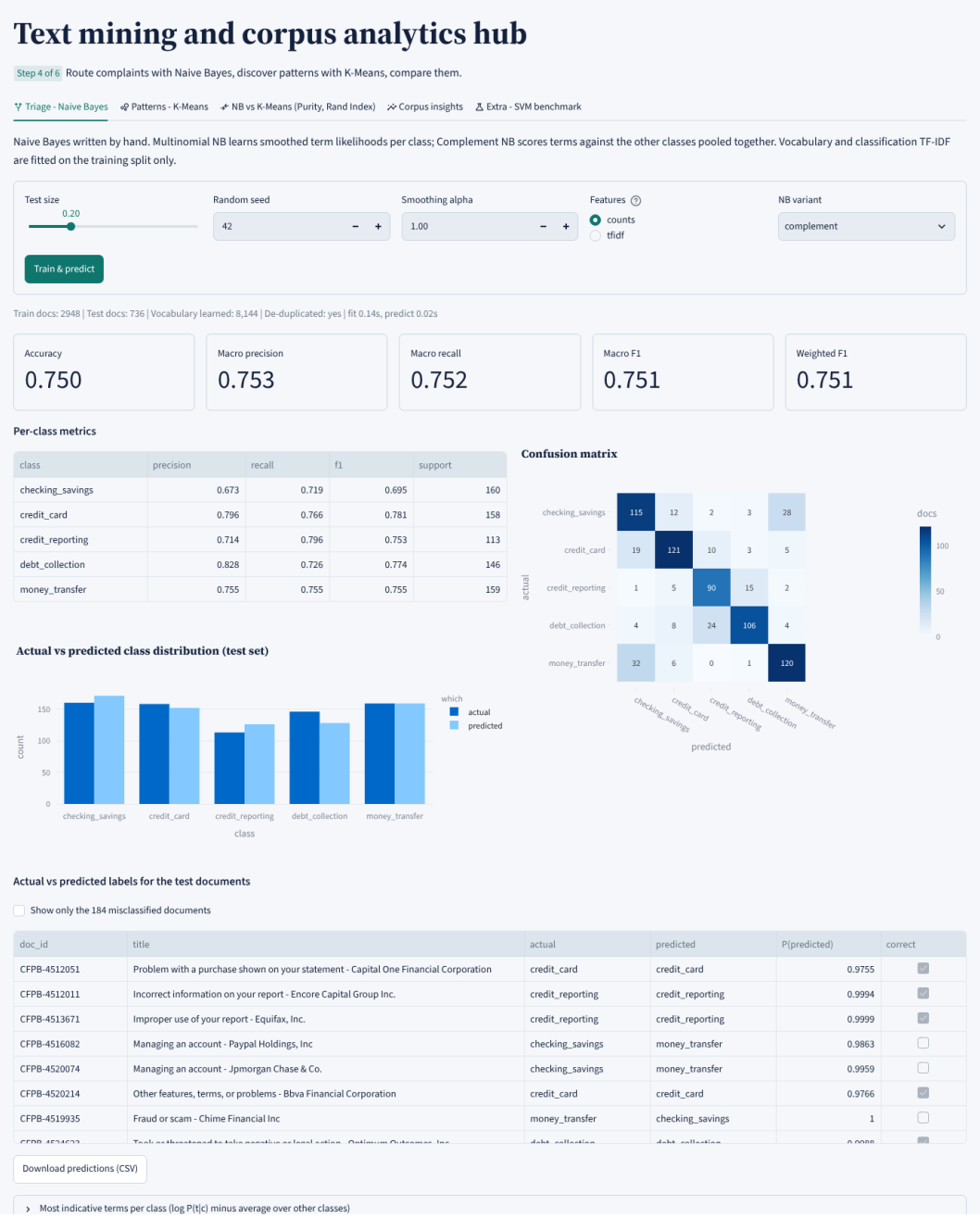


Figure 13: *Text mining hub, Naive Bayes tab: settings, metrics, per-class table, confusion matrix, actual vs predicted and the prediction table*

6. Clustering results (K-Means)

Key result: without seeing any label, spherical K-Means (k = 5) reaches Purity 0.672 and Rand Index 0.788. Three clusters match a product; bank transfers merge with checking, and payment apps / crypto form a cluster of their own.

6.1 Method

src/kmeans.py implements Lloyd's algorithm from scratch on the unit-length tf-idf vectors of the 3,684 de-duplicated complaints. Two versions are available: standard (Euclidean) K-Means and spherical K-Means, which is the default. Spherical K-Means renormalises every centroid to length 1 after the update, so each step maximises the summed cosine similarity of documents to their centroid. For text that is the natural objective: with unit-length centroids, ||x - c||^2 = 2 - 2 cos(x, c), so the reported SSE is twice the summed cosine distance. The steps, and what the app logs for each:

1. **Initialisation:** greedy k-means++. The first centroid is a random document. For each next centroid, 2 + floor(ln k) candidates are drawn with probability proportional to D(x)^2 (squared distance to the closest centroid so far), and the candidate that most lowers the total distance is kept. Plain k-means++ (one candidate) and random initialisation are also available.
2. **Assignment:** every document goes to the nearest centroid, ||x - c||^2 = ||x||^2 + ||c||^2 - 2 x.c.
3. **Update:** each centroid becomes the mean of its documents and, in spherical mode, is scaled back to length 1; an empty cluster is restarted at the worst-served document.
4. **Stopping:** stop when no document changes cluster, or the largest centroid shift is below tol = 1e-4, or after 100 iterations.
5. **Restarts:** run 5 times with different seeds and keep the lowest SSE (chosen without looking at the labels).

6.2 Results (k = 5)

Table 14: *Initial centroids chosen by greedy k-means++ in the best run*

Centroid	Seed document	Title (issue - company)	True label
0	CFPB-5511429	Attempts to collect debt not owed - Transworld Systems Inc	debt_collection
1	CFPB-6098920	Incorrect information on your report - Transunion Intermedia	credit_reporting
2	CFPB-4943338	Managing an account - Jpmorgan Chase & Co.	checking_savings
3	CFPB-5024595	Managing, opening, or closing your mobile wallet account - P	money_transfer
4	CFPB-6198850	Fees or interest - Citibank, N.A.	credit_card

Greedy initialisation already places one seed in each product, which a single random draw rarely does.

Table 15: *K-Means iteration log of the best run*

Iteration	SSE	Docs re-assigned	Max centroid shift	Cluster sizes
1	6709.91	3684	1.1323	[516, 895, 828, 575, 870]
2	5681.12	1007	0.2782	[539, 820, 914, 481, 930]
3	5638.50	461	0.1735	[525, 768, 967, 432, 992]
4	5624.37	210	0.1155	[506, 750, 1016, 405, 1007]
…	…	…	…	…
27	5609.26	1	0.0071	[563, 582, 1182, 356, 1001]
28	5609.25	0	0.0000	[563, 582, 1182, 356, 1001]

The best run stopped after 28 iterations because no document changed cluster; the SSE of the five restarts was 5632.51, 5609.25, 5618.08, 5609.83, 5617.85. Most of the work happens in the first three iterations (SSE 6710 -> 5639); the rest is a slow drift of documents between the largest clusters. Single runs are unstable: over 10 seeds without restarts, purity ranged from 0.448 to 0.631, which is why restarts are used.

Table 16: *Clusters found by spherical K-Means (k = 5)*

Cluster	Theme (our reading of the top terms)	Majority label	Size	Top centroid terms
0	Debt collection letters and validation	debt_collection	563	debt, collection, owe, company, collect, report, letter, collector, validation, agency
1	Credit report disputes, identity theft, FCRA templates	credit_reporting	582	report, section, information, bureau, remove, consumer, delete, credit, inaccurate, theft
2	Bank accounts and bank-to-bank money movement	checking_savings	1182	bank, check, money, transfer, deposit, fund, tell, call, chase, back
3	Payment apps and crypto (PayPal, Coinbase, Venmo)	money_transfer	356	paypal, coinbase, money, fund, venmo, email, help, withdraw, transaction, try
4	Card charges, payments, fees	credit_card	1001	card, charge, payment, fee, pay, credit, call, late, one, balance

Table 17: *Cluster x true label contingency table*

cluster \ label	checking_savings	credit_card	credit_reporting	debt_collection	money_transfer
C0	3	20	45	494	1
C1	8	37	419	112	6
C2	621	99	16	39	407
C3	25	12	0	2	317
C4	143	623	87	83	65

Table 18: *Clustering evaluation with the hand-written Purity and Rand Index*

K-Means evaluation	Value
Purity	0.6716
Rand Index	0.7877
Adjusted Rand Index (extra)	0.3792
Pair counts TP / FP / FN / TN	761,876 / 827,059 / 613,125 / 4,582,026
Total pairs N(N-1)/2	6,784,086
Final SSE / time	5609.25 / 0.54 s

Purity = (1/N) sum over clusters of the size of the majority label = (494 + 419 + 621 + 317 + 623) / 3,684 = 0.6716. Rand Index = (TP + TN) / all pairs = (761,876 + 4,582,026) / 6,784,086 = 0.7877. Both are computed from the contingency table by our own functions in src/metrics.py. What the clusters show:

- **Bank money movement merges with checking.** C2 holds 621 checking and 407 money-transfer complaints: transfers between bank accounts, Zelle and deposits read the same as everyday account problems.
- **Payment apps and crypto form their own cluster.** C3 (356 documents, 317 of them money transfer) is the non-bank side: PayPal, Coinbase, Venmo, withdrawals and e-mail support. K-Means splits money transfer into "through my bank" and "through an app", which the product label does not do.
- **Three clusters are product-like.** C0 (debt collection, 494 of 563), C1 (credit reporting, 419 of 582) and C4 (credit card, 623 of 1001) match a product well. Debt collection and credit reporting, which overlap heavily in vocabulary, are separated.

Table 19: *Cluster assignments, two complaints per cluster (all 3,684 are in results/kmeans_membership.csv and in the membership table on the K-Means tab)*

doc_id	Issue - company	True label	Cluster	Cluster majority
CFPB-4510222	Written notification about debt - Frontline Asset Strat	debt_collection	C0	debt_collection
CFPB-4511553	Incorrect information on your report - Transunion Inter	credit_reporting	C0	debt_collection
CFPB-4508027	Incorrect information on your report - Santander Consum	credit_reporting	C1	credit_reporting
CFPB-4512011	Incorrect information on your report - Encore Capital G	credit_reporting	C1	credit_reporting
CFPB-4511064	Fraud or scam - Paypal Holdings, Inc	money_transfer	C2	checking_savings
CFPB-4511296	Fraud or scam - Moneygram Payment Systems Worldwi..	money_transfer	C2	checking_savings
CFPB-4508204	Fraud or scam - Coinbase, Inc.	money_transfer	C3	money_transfer
CFPB-4507448	Managing, opening, or closing your mobile wallet accoun	money_transfer	C3	money_transfer
CFPB-4506351	Attempts to collect debt not owed - I.C. System, Inc.	debt_collection	C4	credit_card
CFPB-4510320	Problem with a purchase shown on your statement - Bank	credit_card	C4	credit_card

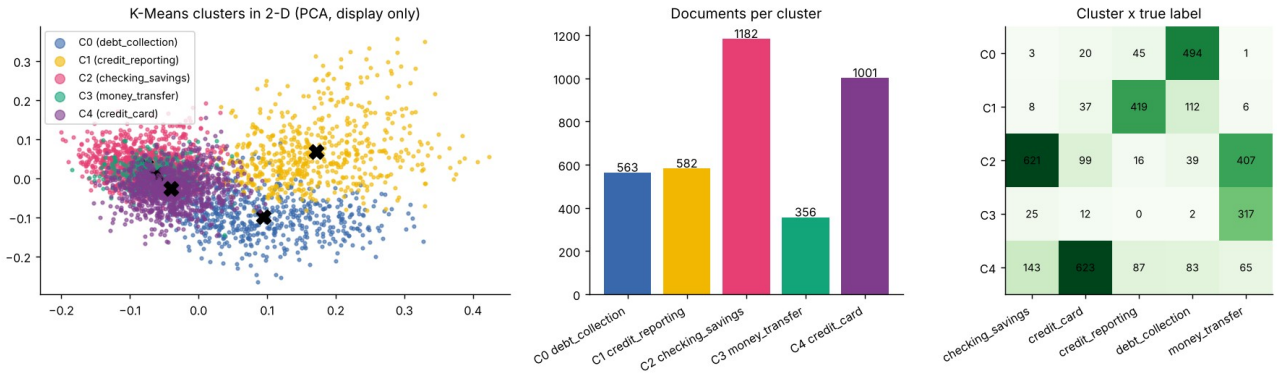


Figure 14: *2-D view of the clusters (PCA for display only, X = centroid), cluster sizes and contingency table*

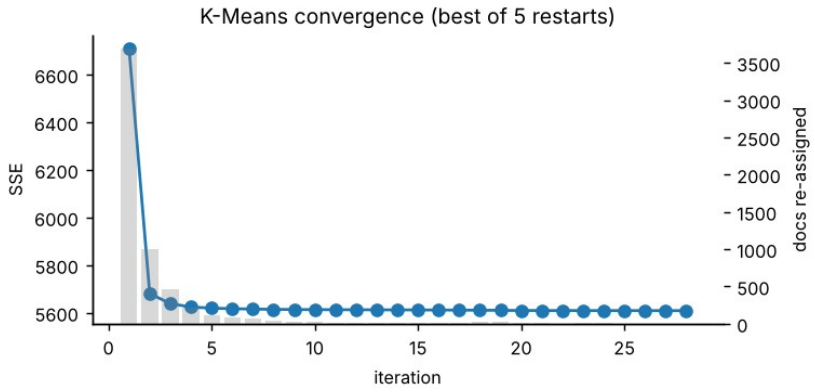


Figure 15: *Convergence of the best run: SSE and documents re-assigned per iteration*

The 2-D plot uses PCA computed from the document-document Gram matrix (double-centred X.X^T, top two eigenvectors). It is only for drawing; K-Means runs on the full vectors. The two components keep under 2% of the variance, so the overlap in the picture overstates the real overlap. The app also offers t-SNE.

6.3 Spherical vs Euclidean K-Means

Table 20: *Effect of the spherical objective and greedy initialisation*

Metric (seed 42, k = 5, 5 restarts)	Euclidean, k-means++	Spherical, greedy k-means++	Change
K-Means purity	0.5831	0.6716	+0.0885
K-Means Rand index	0.7605	0.7877	+0.0272
K-Means ARI	0.3525	0.3792	+0.0266

Table 21: *Stability over three seeds (42, 7, 123), each with 5 restarts*

Objective	Initialisation	Purity (mean +/- sd)	Rand Index	ARI
Euclidean	greedy k-means++	0.615 +/- 0.045	0.761	0.313
Euclidean	k-means++	0.553 +/- 0.050	0.746	0.314
spherical	greedy k-means++	0.645 +/- 0.046	0.779	0.361
spherical	k-means++	0.614 +/- 0.039	0.771	0.361

Both changes help, and they help most together: spherical K-Means with greedy initialisation has the best mean purity and ARI over three seeds. The objective matters more than the initialisation. With Euclidean means, long and short complaints pull centroids towards the corpus average; renormalising keeps each centroid a direction in term space. Results still move with the seed (seed 7 gives a weaker clustering), so the app keeps 5 restarts by default. Euclidean and spherical SSE values measure different objectives and are not compared directly.

6.4 Choice of k

Table 22: *K-Means for k = 2 … 12*

k	SSE	Purity	Rand Index	ARI
2	5746.9	0.360	0.560	0.199
3	5695.0	0.460	0.704	0.294
4	5648.3	0.591	0.759	0.366
5	5609.3	0.672	0.788	0.379
6	5582.7	0.596	0.774	0.286
7	5546.0	0.588	0.774	0.261
8	5529.3	0.557	0.753	0.217
9	5499.7	0.639	0.789	0.260
10	5478.3	0.650	0.794	0.246
11	5452.8	0.579	0.783	0.196
12	5427.2	0.594	0.787	0.195

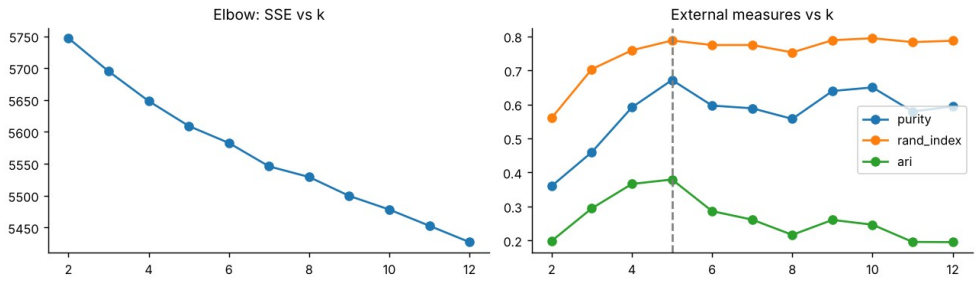


Figure 16: *SSE elbow and external measures as k changes*

The SSE curve falls steadily with no clear elbow, which is typical for sparse text. ARI is highest at k = 5 (0.379), so the number of products is also the best k against the labels. Purity is not monotone here (it peaks at k = 5 and rises again at k = 9-10), because small clusters are easy to keep pure; ARI corrects for chance and drops as soon as real products are split, so it is the better guide. Labels aside, larger k gives useful sub-themes. At k = 8 the clustering separates identity theft on credit reports (report, theft, identity, delete), the FCRA template letters (consumer, furnish, privacy, right, usc), credit-score and loan disputes, and a small Wells Fargo claims group from the rest of checking. For a fraud team the identity-theft and template clusters are more useful than the product labels.

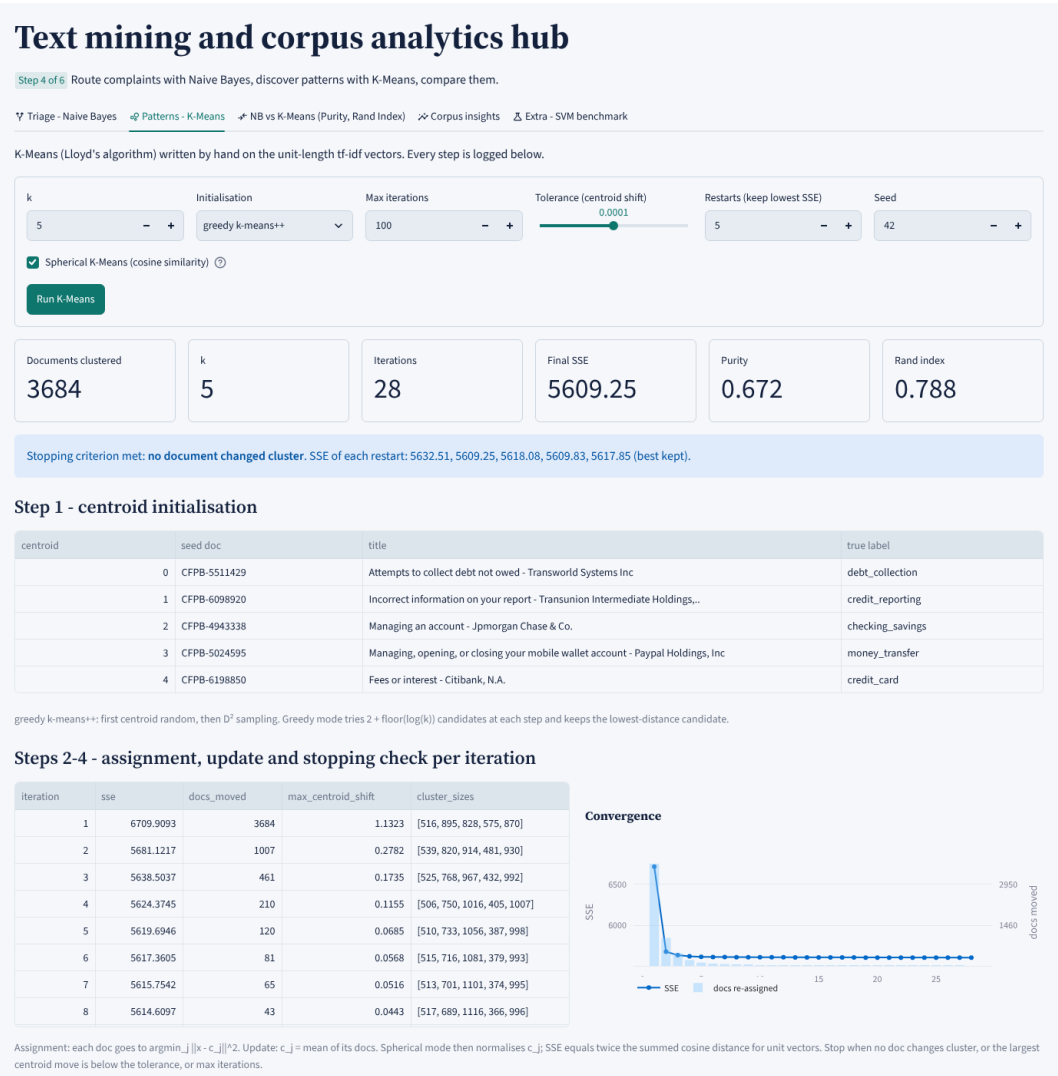


Figure 17: *Text mining hub, K-Means tab: settings, initial centroids, iteration log, convergence chart and stopping criterion*

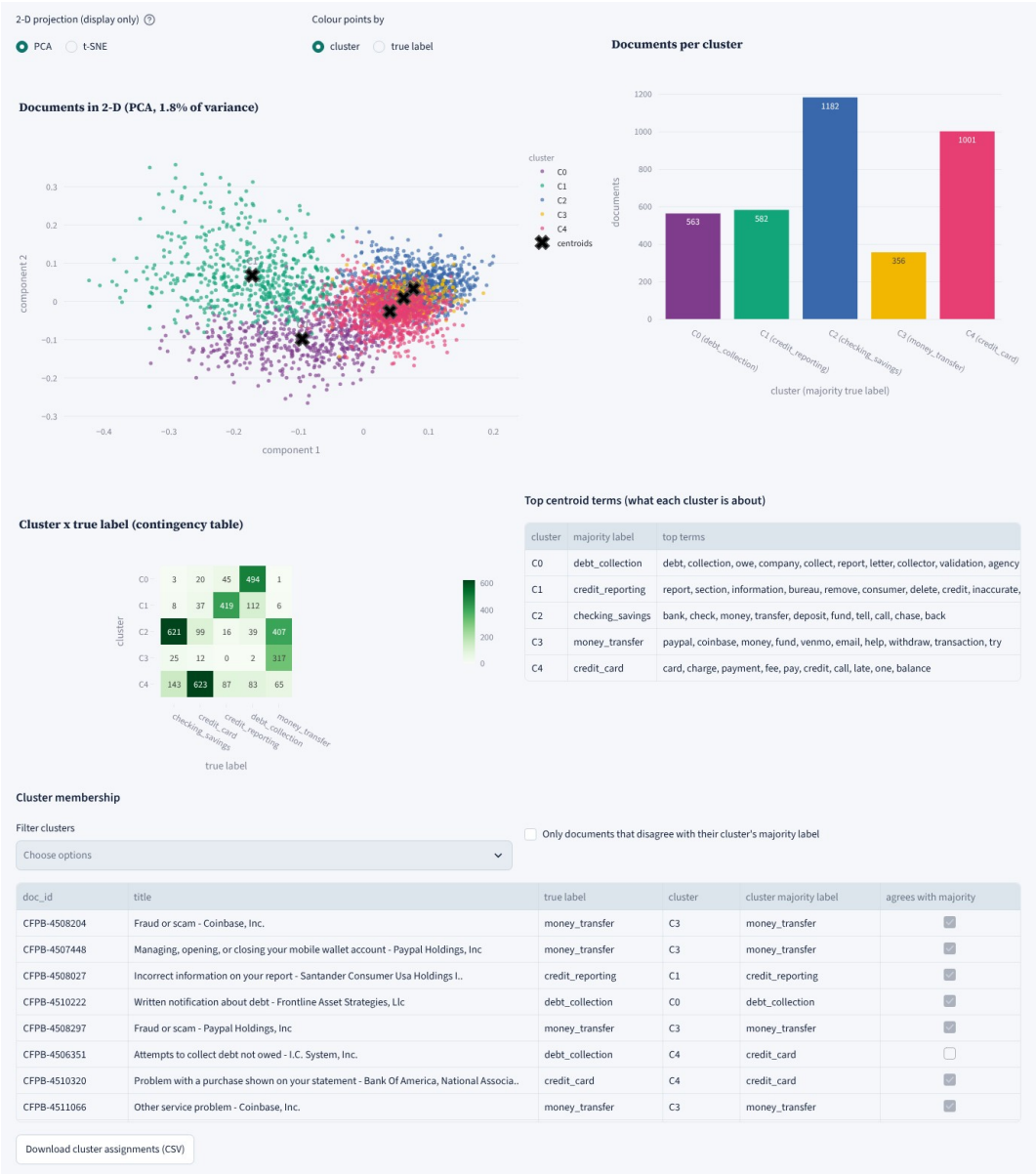


Figure 18: *Text mining hub, K-Means tab: 2-D scatter, cluster sizes, contingency table, top terms and membership table*

7. Comparison of classification and clustering

Key result: on the same 736 test complaints, supervised NB beats unsupervised K-Means by 10.7 points of purity (0.750 vs 0.643) and on Rand Index (0.834 vs 0.774). The labels buy a clear but not overwhelming gain, because the products themselves overlap.

Table 23: *Naive Bayes vs K-Means on the de-duplicated corpus*

Method	Metric	Value
Naive Bayes (supervised)	Accuracy	0.7500
	Macro precision / recall / F1	0.7531 / 0.7524 / 0.7514
	Accuracy, mean of 5 other splits	0.7821
K-Means (unsupervised)	Purity	0.6716
	Rand Index	0.7877
	Adjusted Rand Index	0.3792

Table 24: *Both methods scored on the same footing*

Method	Documents	Accuracy*	Macro-F1	Purity	Rand Index
Naive Bayes (predicted class used as a cluster)	736	0.7500	0.7514	0.7500	0.8341
K-Means, same test complaints	736	0.6427	-	0.6427	0.7745
K-Means, all de-duplicated complaints	3,684	0.6716	-	0.6716	0.7877

* For K-Means, accuracy names each cluster after its majority product, which equals purity and is only meaningful because k equals the number of products. For NB, purity and Rand Index treat the predicted classes as clusters, so both methods can be measured with the same yardstick.

Table 25: *Per-product view*

Product	NB recall	Share of product in a cluster with that majority
checking_savings	0.719	0.776
credit_card	0.766	0.788
credit_reporting	0.796	0.739
debt_collection	0.726	0.677
money_transfer	0.755	0.398

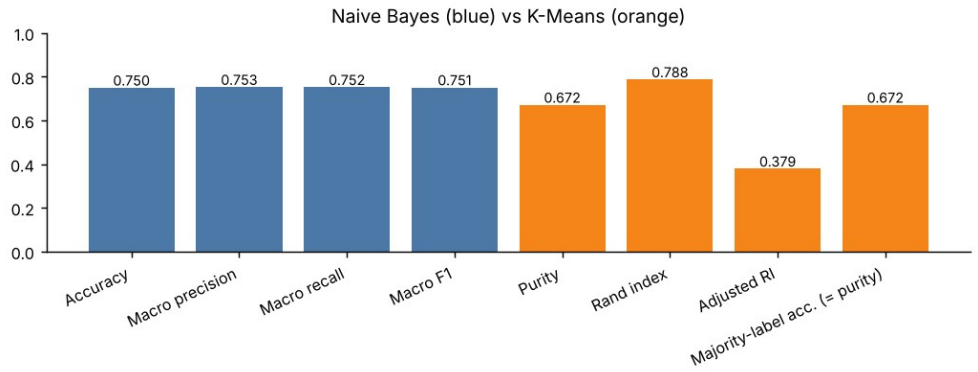


Figure 19: *Side-by-side metrics (blue = Naive Bayes, orange = K-Means)*

- **The supervised model still wins:** NB reaches 75.0% accuracy because it is told the product labels and learns term statistics per product, including company names. K-Means gets 67.2% purity: reading each cluster as its majority product (which is what purity measures) is 7.8 points behind NB. The gap was 17 points with Euclidean K-Means; the spherical objective closed half of it.
- **The metrics are on different scales:** accuracy and F1 judge each complaint's predicted label; purity and RI judge a grouping. RI (0.788) is higher than purity because 4,582,026 of the 6,784,086 pairs are true negatives; ARI (0.379) corrects for chance and is the most honest of the three.
- **Clustering finds different, still useful structure:** K-Means groups by what the text is about. Its clusters (debt letters, credit disputes, bank money movement, app and crypto payments, card charges) are coherent themes that partly cut across the CFPB product taxonomy. Low purity here partly measures the gap between how consumers describe problems and how the form makes them pick a product.
- **Per product:** money transfer is the weakest product for K-Means (only 40% of it sits in its own cluster; most bank transfers fall into the checking cluster), while checking reaches 78%. Naive Bayes is far more even across products (recall 0.72-0.80).

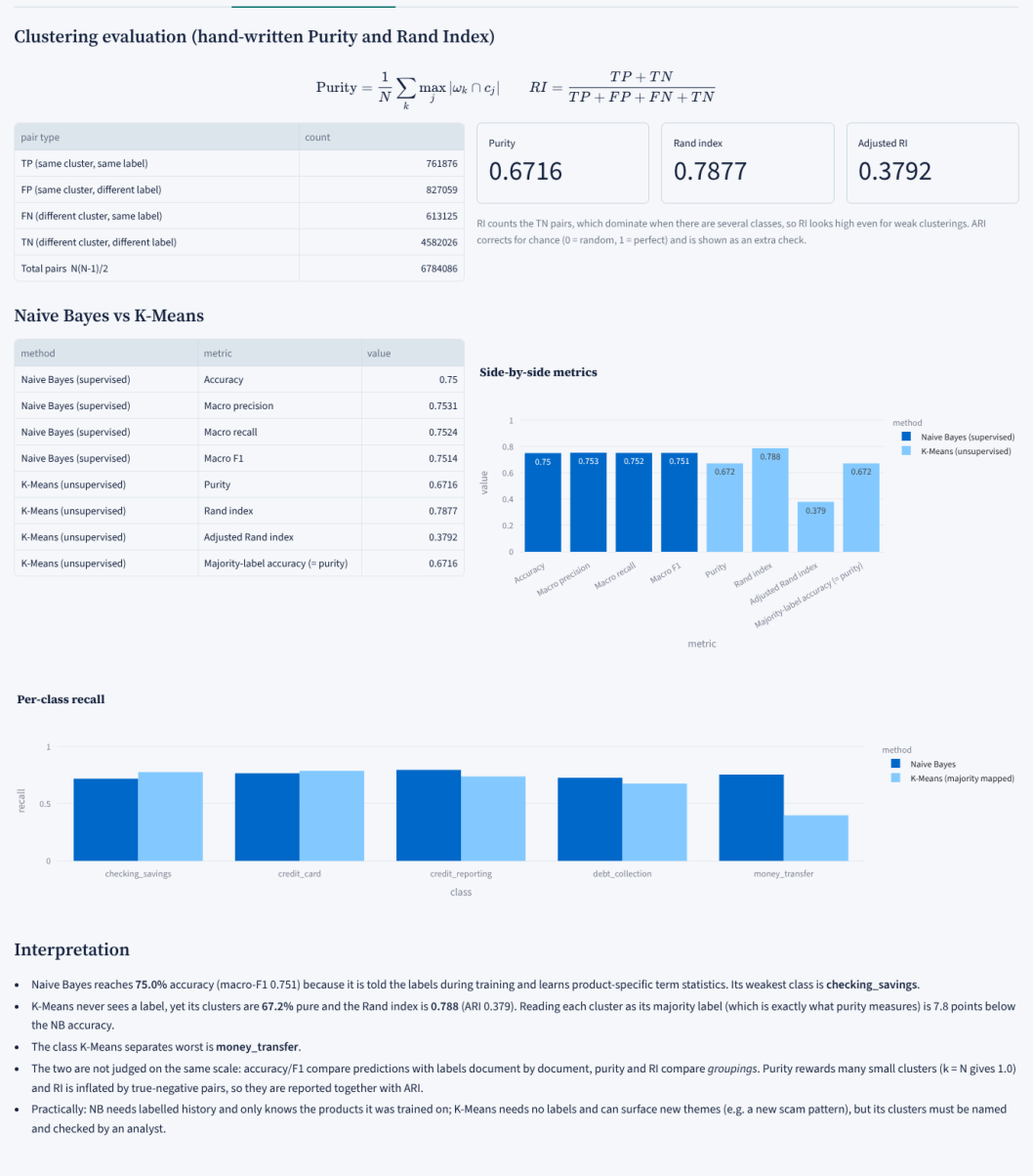


Figure 20: *Text mining hub, NB vs K-Means tab: pair counts, Purity / RI / ARI, side-by-side metrics, per-class recall and interpretation*

8. Recommendation results

Key result: for a complaint picked at random, 62% of its top-5 recommendations are filed under the same product (about three times a random list), and the shared terms explain each match. For fraud cases the list surfaces the same modus operandi (e.g. Zelle impersonation scams) across products.

8.1 Approach: content-based Top-K

We chose content-based recommendation because the corpus has complaint text and metadata but no user interaction data (no reads, clicks or case links), which collaborative filtering needs. src/recommender.py:

1. **Query vector:** the selected complaint's ltc vector, or a free-text case description run through the same preprocessing;
2. **Scoring:** for each query term, walk its weighted posting list and add w(t,q) x w(t,d) to a per-document accumulator. Both vectors are unit length, so the accumulator is the cosine;
3. **Filtering and ranking:** drop the complaint itself; optionally drop near-duplicates of it (cosine >= 0.9) and other products; sort and return the Top-K;

4. **Explanation:** every result is shown as a case card with rank, product badge, issue - company, a cosine bar, doc_id, date, state, the terms that contributed most, and the full text on demand. A score chart and a table (with CSV download) show the same list.

8.2 Example results

Table 26: *Top-5 for CFPB-5156048 "Fraud or scam - Wells Fargo & Company"*

Rank	doc_id	Issue - company	Product	Cosine	Top shared terms
1	CFPB-5090886	Fraud or scam - Early Warning Services, Llc	money_transfer	0.257	caller, zelle, fargo, scam, incident
2	CFPB-5319804	Managing an account - Wells Fargo & Company	checking_savings	0.252	caller, fargo, block, well, answer
3	CFPB-5162703	Fraud or scam - Wells Fargo & Company	money_transfer	0.234	conveniently, fargo, caller, incident, course
4	CFPB-6163129	Fraud or scam - Wells Fargo & Company	money_transfer	0.213	fargo, well, transfer, deny, phone
5	CFPB-5075925	Managing an account - Wells Fargo & Company	checking_savings	0.211	fargo, well, transfer, complete, look

Table 27: *Top-5 for CFPB-4570909 "Problem with a purchase shown on your statement - Capital One Financial Corporation"*

Rank	doc_id	Issue - company	Product	Cosine	Top shared terms
1	CFPB-5921403	Opening an account - Regions Financial Corporati	checking_savings	0.225	region, verify, refuse, use
2	CFPB-6077479	Getting a credit card - Bread Financial Holdings	credit_card	0.215	mile, store, live, fraudulent, charge
3	CFPB-5357581	Advertising and marketing, including promotional	credit_card	0.182	mile, whether, certain, card, use
4	CFPB-4723857	Fraud or scam - Regions Financial Corporation	money_transfer	0.180	region, whether, home, confirm, fraudulent
5	CFPB-6071319	Managing an account - Regions Financial Corporat	checking_savings	0.179	region, certain, purchase, card, transaction

CFPB-5156048 is an impersonation scam: a caller spoofing Wells Fargo talked the customer into a Zelle transfer. Its neighbours are other impersonation scams (shared terms caller, zelle, fargo, scam), which is exactly what an investigator wants: a set of similar modus-operandi cases. Two of the five are filed under checking rather than money transfer, which shows the product-overlap problem again. CFPB-4570909 (fraudulent card charges) gets weaker results: the strongest shared terms are "region" (from Regions Bank, and the customer's region) and "mile", so the bag-of-words similarity is driven by incidental words rather than the fraud pattern.

Table 28: *CFPB-6103235 (a credit-reporting template letter) with the duplicate filter OFF and ON*

Rank	doc_id	Issue - company	Cosine	Duplicate filter
1	CFPB-6138560	Improper use of your report - Transunion Interme	1.000	OFF
2	CFPB-5211539	Improper use of your report - Equifax, Inc.	0.621	OFF
3	CFPB-5935054	Incorrect information on your report - Experian	0.621	OFF
1	CFPB-5435343	Improper use of your report - Equifax, Inc.	0.621	ON
2	CFPB-5211539	Improper use of your report - Equifax, Inc.	0.621	ON
3	CFPB-5871148	Improper use of your report - Experian Informati	0.621	ON

With the filter off, the first result for CFPB-6103235 is a word-for-word copy (cosine 1.000) filed against another credit bureau; the following ones are the same template at 0.62. With the filter on, the copy disappears. Template letters are a case where the recommender is right that the documents are similar, but the result is not useful.

Table 29: *Top-5 for the free-text query "Someone pretending to be from my bank told me to send money through Zelle and now the bank says it was authorized and will not refund me"*

Rank	doc_id	Issue - company	Product	Cosine	Top shared terms
1	CFPB-5846253	Fraud or scam - Early Warning Services, Llc	money_transfer	0.362	zelle, refund, money, bank
2	CFPB-5826356	Fraud or scam - Early Warning Services, Llc	money_transfer	0.354	zelle, pretend, refund, someone, bank
3	CFPB-4532493	Managing, opening, or closing your mobile wallet acc	money_transfer	0.332	zelle, bank, tell, send
4	CFPB-6256171	Unauthorized transactions or other transaction probl	money_transfer	0.317	zelle, money, bank
5	CFPB-5829877	Fraud or scam - Early Warning Services, Llc	money_transfer	0.268	zelle, someone, money, bank, send

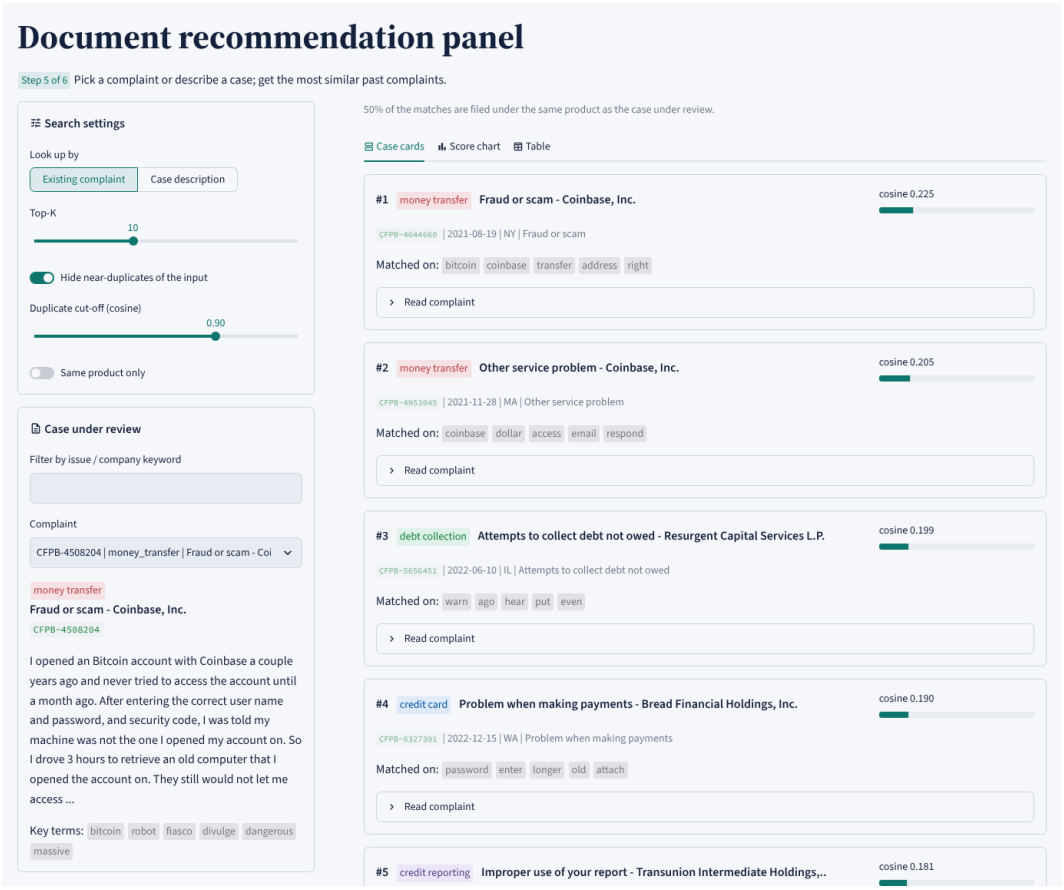


Figure 21: *Document recommendation panel: search settings, the case under review and the Top-K case cards*

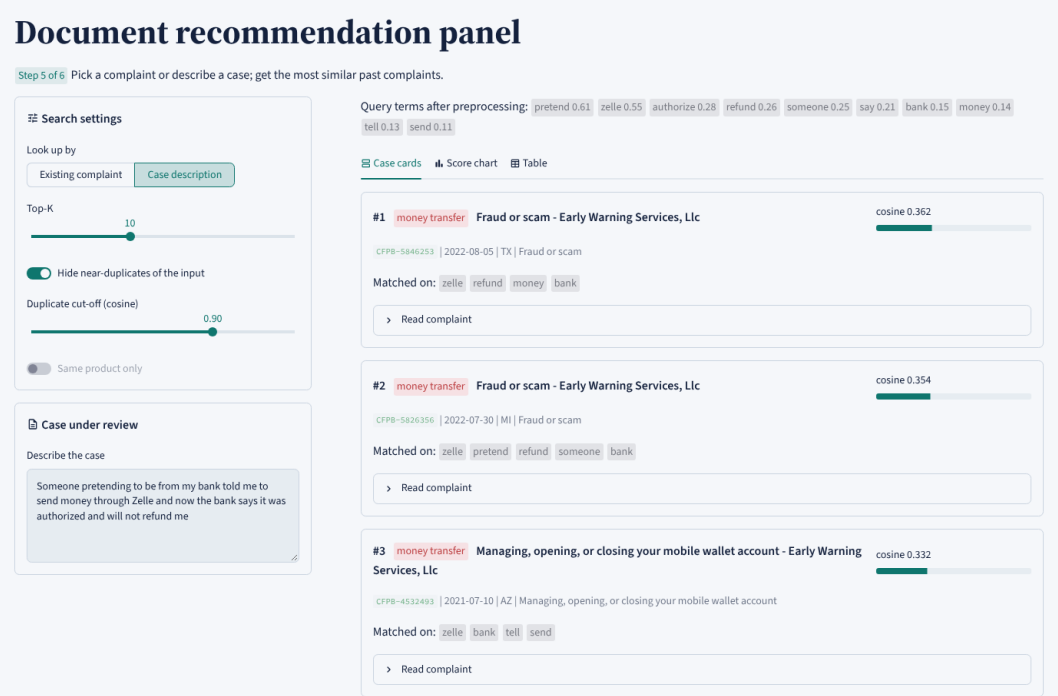


Figure 22: *Document recommendation panel in free-text mode with the query term weights*

8.3 Quality

Without investigator feedback we use "same product" as a relevance proxy and average precision@K over every complaint as the query:

Table 30: *Recommendation precision@K over all 4,000 complaints*

K	precision@K (all candidates)	precision@K (near-duplicates removed)
1	0.671	0.662
3	0.639	0.631
5	0.620	0.613
10	0.595	0.588
15	0.577	0.572
20	0.565	0.561

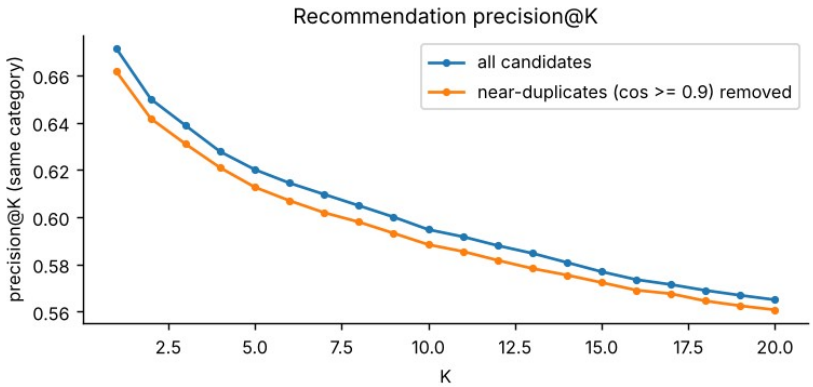


Figure 23: *precision@K with and without near-duplicates*

About 6 of the top 10 recommendations come from the same product (P@10 = 0.595). That is far above the 0.2 a random list would give, but it also shows how much the products overlap in vocabulary. For an investigator, "same product" is a weak proxy: the Zelle scam example above is useful even when a neighbour is labelled checking. Of the 393 complaints that have a duplicate, 84.5% get it as their first recommendation if no filter is applied.

8.4 Limitations of the method on this dataset

- Bag of words: "scammer" and "fraudster", or "not mine" and "unauthorized", are unrelated terms; phrasing matters more than meaning.
- Incidental words (place names, bank names that are also common words such as "Regions") can dominate the score.
- The CFPB masks names, dates, amounts and account numbers as XXXX, so the strongest linking signals for fraud investigation (same phone number, same payee, same amount) are not available.
- Template complaints fill the list unless the duplicate filter is on.
- No personalisation or feedback: every investigator gets the same list; relevance is only approximated by product.

9. Inference and discussion

All numbers come from the experiment scripts (results/ folder); the same experiments can be run live on the Performance analytics page. To separate real effects from split and seed noise, Naive Bayes is averaged over 5 stratified splits and K-Means over 3 seeds (each with 5 restarts). In Q1 every setting is scored on the same 3,684 de-duplicated complaints and the same splits, so only the preprocessing changes.

Q1. How did the choice of preprocessing affect the vocabulary, classification, clustering and recommendations?

Table 31: *Effect of preprocessing and weighting choices (Complement NB, spherical K-Means; every setting uses the same 3,684 de-duplicated complaints and the same splits)*

Setting	Index terms	NB accuracy	K-Means purity	K-Means ARI	P@10	Top-10 overlap with D
A. minimal (lowercase only)	7,878	0.776 +/- 0.010	0.566 +/- 0.016	0.300	0.568	0.55
B. + redactions, numbers, stop-words removed	7,142	0.782 +/- 0.007	0.627 +/- 0.028	0.364	0.575	0.64
C. B + stemming	4,492	0.781 +/- 0.012	0.625 +/- 0.048	0.339	0.572	0.80
D. B + lemmatisation (default)	5,077	0.782 +/- 0.010	0.645 +/- 0.038	0.361	0.571	1.00
E. D + bigrams	46,802	0.789 +/- 0.011	0.599 +/- 0.039	0.345	0.572	0.44
F. D but XXXX redactions kept	5,083	0.783 +/- 0.007	0.657 +/- 0.026	0.355	0.571	0.96
G. D but original case kept	6,943	0.776 +/- 0.007	0.548 +/- 0.080	0.288	0.561	0.65
H. D, NB on tf-idf features	5,077	0.749 +/- 0.008	0.645 +/- 0.038	0.361	0.571	1.00
I. D, raw tf (NB on tf-idf)	5,077	0.763 +/- 0.011	0.615 +/- 0.005	0.356	0.599	0.46
J. D, no idf (NB on tf-idf)	5,077	0.771 +/- 0.012	0.608 +/- 0.008	0.319	0.584	0.37

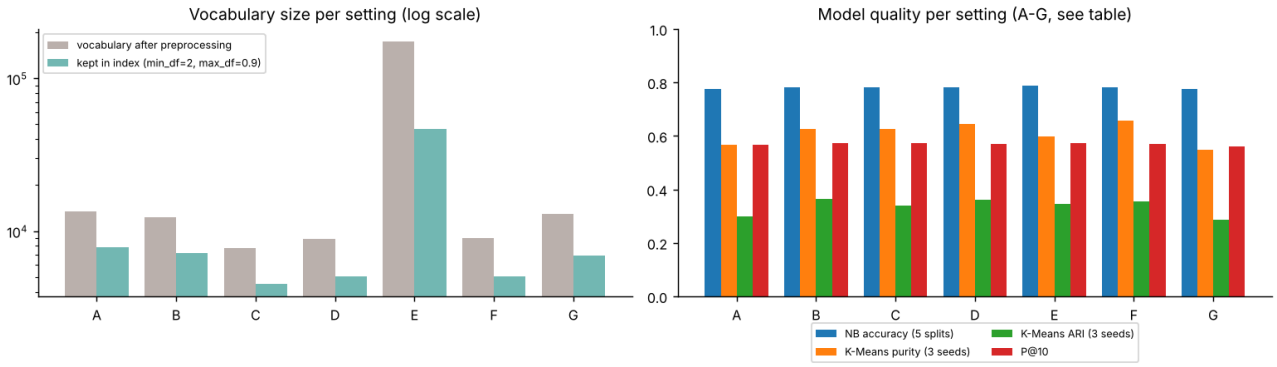


Figure 24: *Vocabulary size and model quality for settings A-G*

- **Vocabulary:** cleaning and stop-words (A -> B) cut tokens from 887,711 to 379,494; stemming gives the smallest index (4,492 terms), lemmatisation keeps more readable terms (5,077); bigrams multiply the vocabulary by nine (46,802).
- **Classification:** Naive Bayes barely moves. Settings A-G are all between 0.776 and 0.789, about one split-to-split standard deviation (0.01). NB learns per-class statistics, so frequent words that appear in every product mostly cancel out. Bigrams (E) give the best NB accuracy (0.789), because phrases such as credit_report or

identity_theft separate products, but at nine times the vocabulary. The minimal setting (A) and keeping the original case (G) are the lowest (0.776 and 0.776).

- **Clustering:** K-Means is sensitive. Keeping the original case (G) is the worst setting (purity 0.548, ARI 0.288, against 0.645 / 0.361 for D): "Bank" and "bank", "ZELLE" and "zelle" become different terms, so similar complaints share fewer weights. Without stop-words and cleaning (A), ARI drops to 0.300 because common words dominate the similarities. Stemming (C) lowers ARI to 0.339 and bigrams (E) to 0.345 with 47k sparse dimensions. The weighting matters too: no idf (J) gives ARI 0.319. Settings B and F are slightly above D on one measure each, within the seed noise.
- **Recommendations:** precision@10 stays between 0.56 and 0.60 in every setting, but the actual lists change a lot. Compared with D, the share of top-10 results that stay the same is 80% with stemming, 64% without lemmatisation, 55% for the minimal setting, 44% with bigrams and only 37% without idf. Preprocessing therefore changes which cases an investigator sees, even when the category-level score looks unchanged.
- **Redactions and idf:** keeping the XXXX masks (F) hardly changes anything (NB 0.783, top-10 overlap 0.96). The token "xxxx" appears in 85% of complaints, so its idf is log10(4000/3417) = 0.07 and its weight is close to zero: idf neutralises the redaction noise on its own. Removing the masks still matters for duplicate detection and readability.
- **Features for NB:** for NB, raw counts beat tf-idf features (0.782 vs 0.749), the same result as the validation search in Section 5.1. Multinomial-type NB is built for counts, and length normalisation removes information it uses.
- **Normalisation errors:** WordNet lemmatisation without part-of-speech tags makes visible errors on this corpus: "Wells" (Fargo) becomes "well" and "boss" becomes "bos". Stemming makes others ("business" -> "busi").

Conclusion: cleaning, stop-word removal, lower-casing and log-tf with idf are the choices that matter, mostly for clustering and for which cases get recommended. Stemming vs lemmatisation is mainly about readability; bigrams help classification a little but hurt clustering and cost nine times the vocabulary. We kept D as the default because it is good across all three tasks and its terms are readable in cluster and recommendation explanations.

Q2. How can duplicates affect classification, clustering and recommendation? How did we detect and handle them?

Detection (Section 2.3): an MD5 hash of the normalised text for exact copies, and TF-IDF cosine >= 0.85 for near copies, with 5-shingle Jaccard shown as a second opinion. Handling: a toggle on the Data ingestion page keeps the first complaint of every duplicate group (union-find over the pairs) and removes the other 316 before classification and clustering. The recommendation panel has its own filter that hides results with cosine >= 0.9 to the query. Measured effect:

Table 32: *Impact of duplicates*

Measure	With duplicates	De-duplicated
Documents	4,000	3,684
Test complaints with an exact copy in the training set (avg per split)	15.4	0
NB accuracy on those leaked test complaints	0.864	-
NB accuracy on the other test complaints	0.7835	0.7806
NB accuracy, mean of 10 splits	0.7850	0.7806
K-Means purity / ARI (mean of 3 seeds)	0.6051 / 0.3432	0.6448 / 0.3609
Recommendation P@1	0.6715	0.6469
Complaints whose top-1 recommendation is a copy	8.3%	0.0%

- **Classification:** with duplicates, about 15 test complaints per split have an exact copy in training, and NB gets 86.4% of them right against 78.3% for the rest. That is leakage: the model is partly tested on text it has seen. NB is not a lookup table, so one copy does not guarantee a correct answer. Overall accuracy moves only slightly (0.7850 vs 0.7806) because exact copies are few, but the reported number is flattered in the same direction. The bigger problem is the 236 near-identical templates filed under two different products: they put contradictory training examples into the data and cap the accuracy any text classifier can reach.

- **Clustering:** copies count several times in a centroid and in the purity / RI sums. Here duplicates lower purity (0.605 vs 0.645) and ARI (0.343 vs 0.361). The 233 extra credit-reporting template copies give the template wording more weight in a centroid than the product itself. With more copies, one campaign can take over a cluster.
- **Recommendation:** duplicates hurt most here. Without the filter, 84.5% of complaints that have a copy get the copy as their first recommendation, which tells an investigator nothing new. P@1 also looks better with duplicates (0.671 vs 0.647) because a copy almost always has the same product, so duplicates flatter the evaluation as well.

Table 33: *Sensitivity to the near-duplicate threshold (K-Means seed 42, 5 restarts)*

Cosine cut-off	Near pairs	Docs removed	K-Means purity	Rand Index	ARI	P@5
0.80	3,950	354	0.634	0.777	0.376	0.593
0.85	3,379	316	0.672	0.788	0.379	0.596
0.90	2,677	266	0.601	0.763	0.341	0.600
0.95	2,030	226	0.628	0.779	0.391	0.604

- **Choice of threshold:** moving the cut-off from 0.95 to 0.80 removes between 226 and 354 complaints. K-Means purity moves between 0.60 and 0.67, which is the same size as its seed-to-seed variation (Section 6.3), so the clustering does not hinge on the cut-off. P@5 rises slightly as the cut-off rises, because more same-product copies stay in the pool and flatter the score. We use 0.85: it catches template letters whose masked parts differ, without merging distinct complaints about the same issue.
- **Business view:** for a complaints team, a burst of template complaints is itself a signal (a credit-repair company campaign). So duplicates should be grouped and counted, not just deleted.

Q3. The recommendation approach, and content-based vs collaborative

Our system is content-based: a complaint is recommended because its words are similar to the selected complaint or case description (Section 8).

Table 34: *Content-based vs collaborative recommendation*

Aspect	Content-based (ours)	Collaborative
Data needed	Item content (complaint text, metadata)	User-item interactions (which cases each investigator opened, linked or escalated)
How similarity is found	Cosine between item feature vectors	Users with similar histories, or items used by the same users (item-item CF, matrix factorisation)
New items (cold start)	Works immediately: a new complaint can be matched as soon as it is indexed	Fails until enough people interact with the item
New users	Works without history	Fails without history
Explanation	Easy: shared terms are shown	Harder: "investigators who opened this also opened ..."
Diversity	Low; returns near-copies and the same wording	Higher; can link cases worded differently but handled together
Weak points	Vocabulary mismatch, incidental words, templates	Sparsity, popularity bias, needs many users, privacy
Preferable when	Items are text-rich, new ones arrive constantly, no interaction logs (our case)	Large user base with long histories, items hard to describe by content (e.g. a consumer finance app recommending articles)

For complaints and fraud cases, every case is new and must be matched on the day it arrives, so content-based is the right starting point. Once the team logs which past cases investigators link to a new one, a hybrid (content score blended with an item-item "linked together" score) would add links that wording alone misses, e.g. two scams described differently but run by the same fraud ring.

Q4. How do text mining, classification, clustering and recommendation together help domain experts decide?

Table 35: *Value of each component for the complaints and fraud team*

Component	What it gives the analyst	Decision it supports
Text mining / corpus analytics	Fraud share per product, top companies, monthly volume, top terms per product, term spread (e.g. 'zelle'), keyword in context	Where to put investigators; which companies and products drive fraud complaints; spotting volume spikes
Classification (NB)	Routes a complaint to a product team with ~78% accuracy; the predicted probability flags borderline cases	Automatic triage; analysts review only low-confidence or cross-product complaints
Clustering (K-Means)	Groups complaints by theme without labels; centroid terms name the theme	Discover patterns that cut across products (app and crypto payments, identity theft, template campaigns); check whether the product taxonomy still fits
Recommendation	Similar past complaints with scores and shared terms, duplicates hidden	Faster investigations: see how similar scams were handled, spot repeat modus operandi

The components also check each other. NB's checking vs money-transfer confusions and K-Means putting bank transfers into the checking cluster point to the same fact: P2P scams do not respect product boundaries, so a fraud team should own them across products. K-Means also shows that "money transfer" is really two problems, bank transfers and app or crypto wallets, which need different investigators. The template cluster found without labels at k = 8 tells the team that a share of credit-reporting volume is a campaign, not independent customer pain. The recommender turns one confirmed scam into a list of similar cases to review.

Q5. Naive Bayes vs K-Means: key findings, limitations and improvements

Key findings

- Naive Bayes (Complement, chosen on validation splits): accuracy 0.750 on the seed-42 split (0.782 +/- 0.010 over 5 other splits), macro-F1 0.751. Errors are concentrated between products that share vocabulary: checking vs money transfer, and debt collection vs credit reporting. The gain over Multinomial NB (+2.0 points) is consistent across splits but not significant on one split (McNemar p = 0.063).
- K-Means (spherical, greedy k-means++, k = 5): purity 0.672, Rand Index 0.788, ARI 0.379, up from purity 0.583 with standard Euclidean K-Means. Clusters follow themes rather than products: bank transfers merge with checking, and app and crypto payments form their own cluster.
- Supervised learning is still about 8 points better and far more stable; K-Means depends strongly on initialisation (single-run purity 0.45-0.63).
- Preprocessing barely matters for NB but matters for K-Means and for which cases are recommended; idf alone neutralises the XXXX noise.
- Duplicates (template campaigns) mainly damage recommendations and inflate evaluation; they must be removed before splitting, and are worth counting as a signal.

Limitations

- Five products, 4,000 complaints, Jul 2021 - Dec 2022 from an archived snapshot; real volumes are far larger and the taxonomy changes over time.
- Labels are the product the consumer chose on the form, which is itself noisy.
- Bag-of-words features: no phrase meaning, synonyms or context; company names do much of the work for NB.
- K-Means needs k and prefers clusters of similar spread; RI over-rewards true negatives.
- Repeated 80/20 splits overlap, and the reference split was looked at during development, so the scores are a stability check rather than an unbiased deployment estimate; a later, chronological test set would be needed.
- Recommendation relevance is approximated by product; no investigator feedback.
- Everything is in memory (full 4,000 x 4,000 cosine matrix), which would not scale to millions of complaints.

Suggested improvements for real-world deployment

- Use issue and sub-issue (e.g. "Fraud or scam") as an additional, finer label, and train a fraud / not-fraud classifier next to product routing.

- Mask company names in training or add them as separate metadata features, so the text model learns the problem, not the company.
- Add dense sentence embeddings (or LSA) for clustering and recommendation to handle synonyms; keep tf-idf terms for explanations.
- Calibrate the classifier's probabilities and send low-confidence complaints to a human; retrain on analyst corrections.
- Use MinHash / LSH for duplicate detection at scale and treat template bursts as a monitored signal.
- Collect investigator link / escalation logs to move to a hybrid recommender, and measure real relevance with them.

10. Extra: linear SVM benchmark (beyond the brief)

This section is an extra and is not part of the required pipeline. The brief asks for Naive Bayes, which is the classifier used everywhere above. To see how much a discriminative model gains on the same data, we trained scikit-learn's LinearSVC on top of our own hand-written TF-IDF features (src/classifier.py). The TF-IDF weighting, vocabulary and df filters are still ours; only the SVM optimiser comes from the library.

- two feature views, both log-tf x idf with L2 normalisation, min_df = 2, max_df = 90%, fitted on training text only: lemmatised unigrams + bigrams + trigrams (C = 1), and original word forms, unigrams + bigrams (C = 0.3);
- one linear SVM per view (one-vs-rest); the decision margins of the two are averaged and the product with the largest average wins;
- the pair of views and C values was chosen on three validation splits inside the training part (mean validation accuracy 0.804), then evaluated once on the same seed-42 test set as NB.

Table 36: *Classifier comparison on identical splits*

Model	Accuracy, seed 42	Macro-F1, seed 42	Mean accuracy, seeds 0-4
Multinomial NB (manual)	0.7296	0.7314	0.7652
Complement NB (manual, default)	0.7500	0.7514	0.7821
Two-view linear SVM (scikit-learn, extra)	0.7935	0.7936	0.8082

Table 37: *SVM per-class metrics (seed-42 test set)*

Product	Precision	Recall	F1	Support
checking_savings	0.756	0.756	0.756	160
credit_card	0.786	0.835	0.810	158
credit_reporting	0.811	0.761	0.785	113
debt_collection	0.838	0.815	0.826	146
money_transfer	0.787	0.792	0.790	159

The SVM is 4.3 points more accurate than Complement NB on the reference split and 2.6 points on average. F1 improves for every product, most for checking and debt collection, the two products NB found hardest. It does not remove the checking vs money-transfer confusion (26 and 25 errors), which confirms that this confusion is in the labels rather than in the model. Naive Bayes remains the required, fully hand-written model; it trains in a fraction of a second, its term weights are directly readable, and it is within 3-4 points of the SVM.

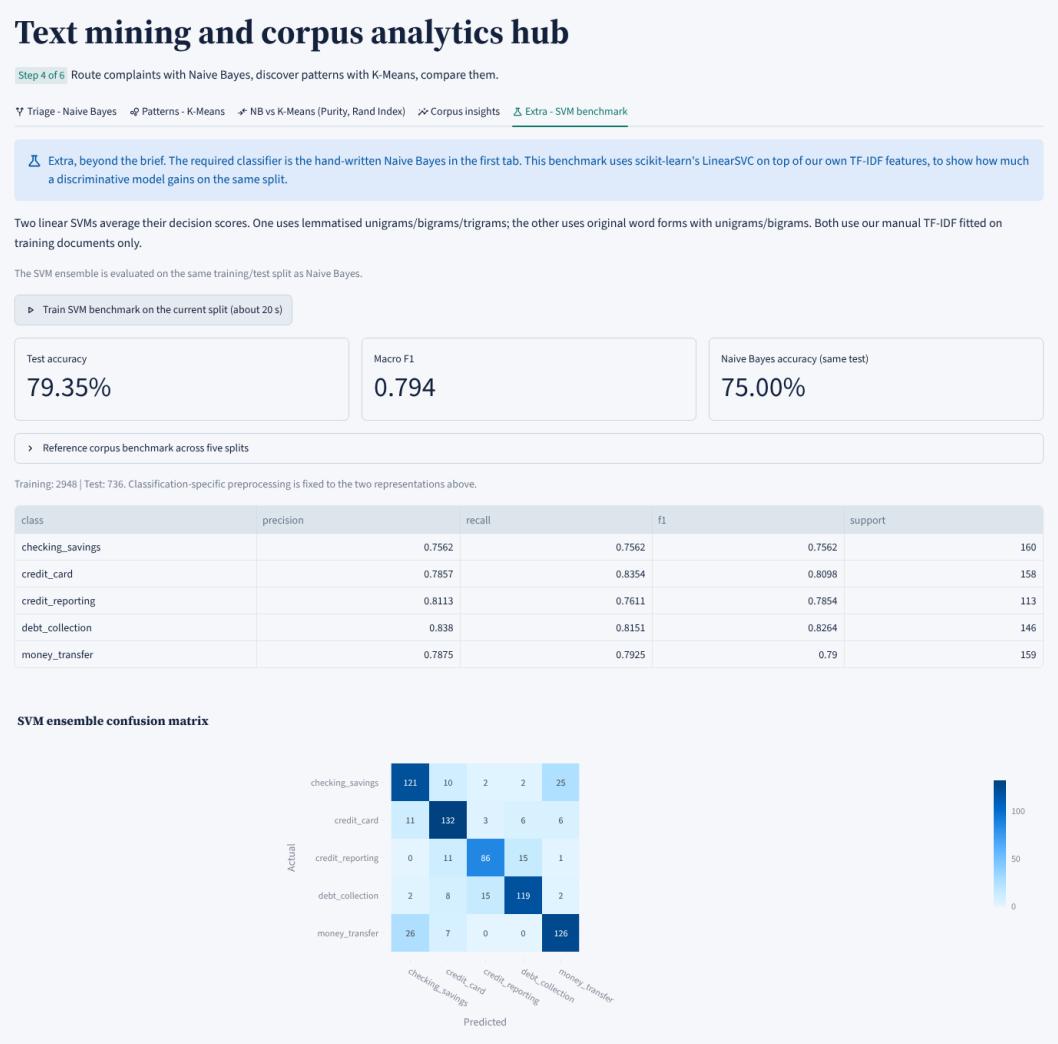


Figure 25: *Text mining hub, Extra - SVM benchmark tab, clearly marked as beyond the brief, trained on the same split as Naive Bayes*

11. Limitations and improvements for deployment

The main limitations and improvements are listed under Q5. On the engineering side: the app rebuilds the index for every configuration change (cached per configuration, a few seconds for this corpus). The NLTK resources must be downloaded once (the app falls back to a built-in stop-word list and the stemmer when they are missing). Uploaded datasets must fit in memory. Secondary experiment numbers (ablation, stability) can differ in the third decimal place between machines because of floating-point ties; the headline numbers are reproducible.

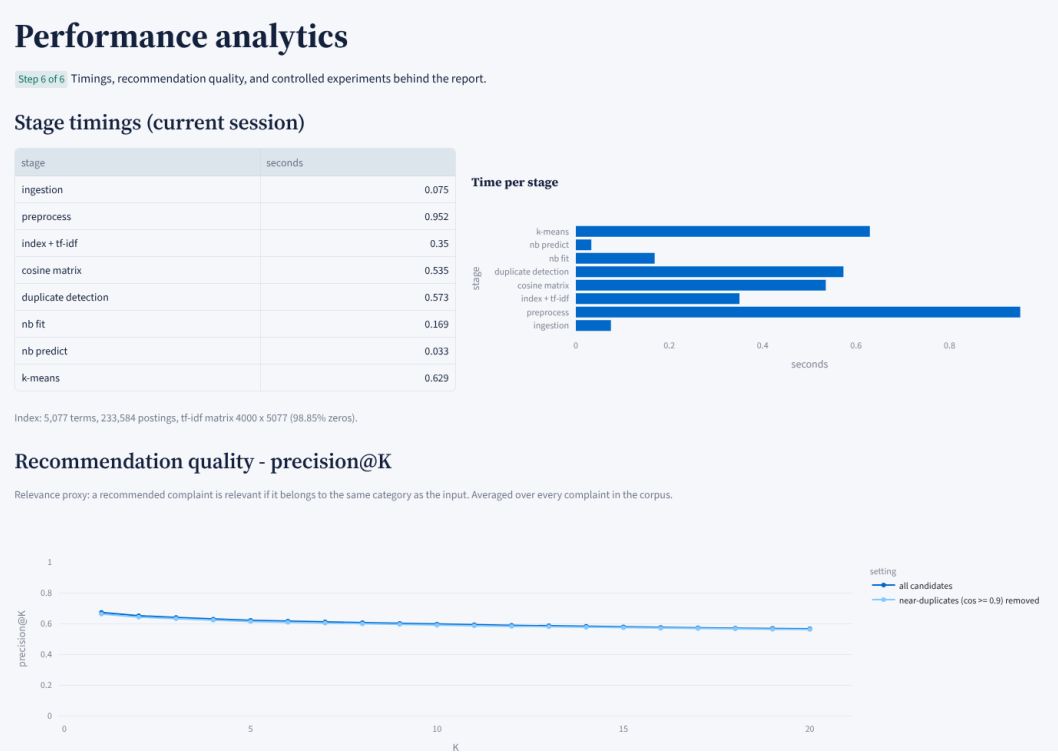


Figure 26: *Performance analytics: stage timings and precision@K*

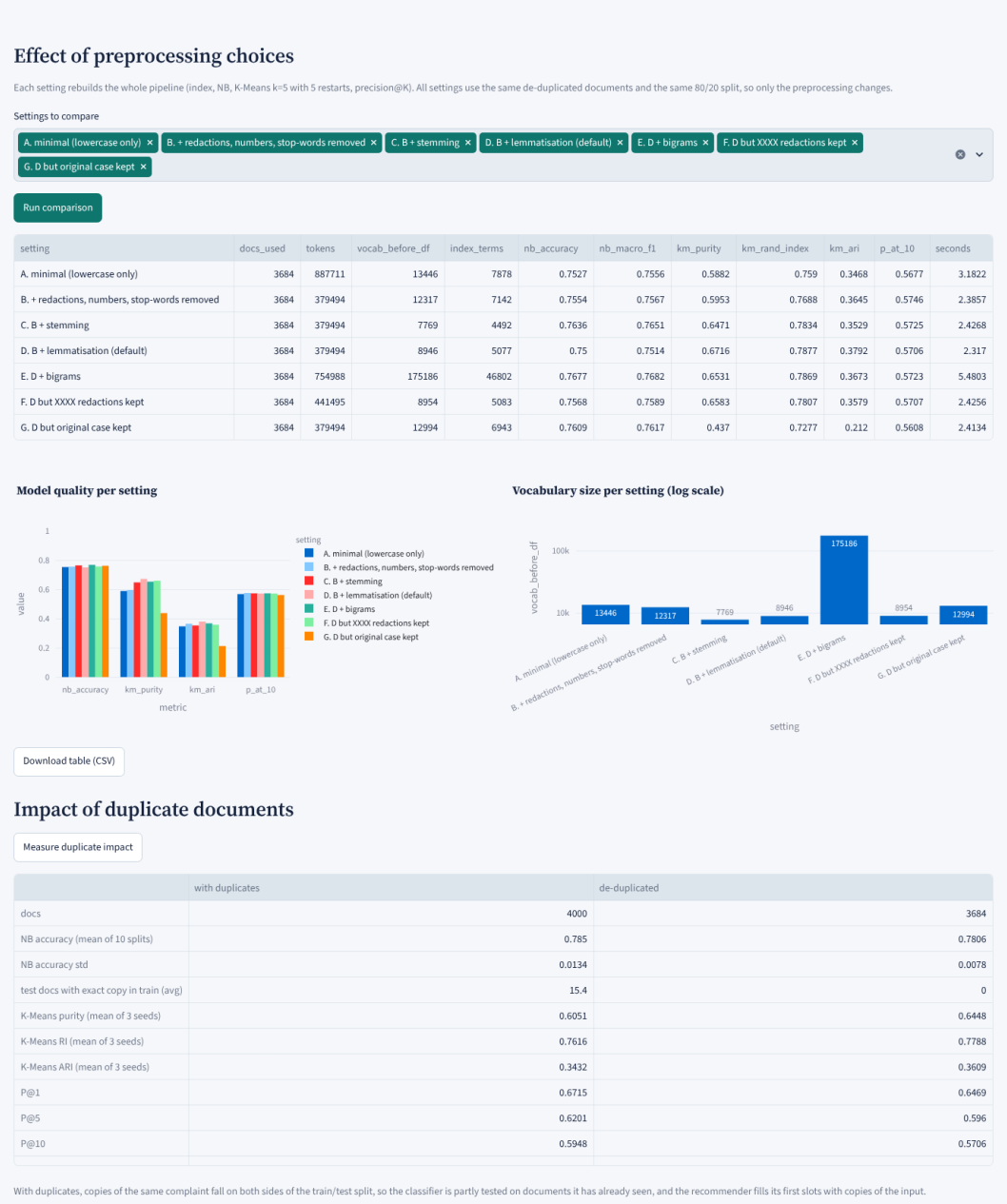


Figure 27: *Performance analytics: preprocessing comparison and duplicate impact computed live*



k	SSE	purity	rand index	ARI	iterations
2	5746.9065	0.3602	0.5605	0.1988	8
3	5695.0041	0.4596	0.7035	0.2944	12
4	5648.3489	0.5912	0.7589	0.3661	22
5	5609.252	0.6716	0.7877	0.3792	28
6	5582.6743	0.5964	0.7744	0.2862	25
7	5545.9764	0.5885	0.7745	0.2612	15
8	5529.2865	0.5573	0.7527	0.2165	26
9	5499.6635	0.6393	0.7885	0.2604	29
10	5478.252	0.6498	0.7944	0.2464	23

SSE always falls as k grows, so the elbow is weak on sparse text. Purity can also rise with k because small clusters are easier to keep pure, so it can't pick k on its own; ARI corrects for chance and drops once real topics get split.

Figure 28: *Performance analytics: choosing k for K-Means*

12. Evidence of execution in the BITS Virtual Lab

The application was installed and run on the BITS Virtual Lab (g4.prayogshala.bits-pilani.ac.in, Linux machine ip-10-3-19-230, Python 3.12) on 3 November 2026 with the commands below, and opened in the Lab's Firefox at localhost:8501. Every number shown in the Lab matches this report (e.g. NB accuracy 0.750, Purity 0.6716, Rand Index 0.7877). The original screenshots are in the virtual_lab/ folder of the submission.

```
python -m venv .venv-linux && source .venv-linux/bin/activate
pip install -r requirements.txt
python -m nltk.downloader stopwords wordnet omw-1.4
python -m streamlit run app.py --server.address 0.0.0.0 --server.port 8501
```

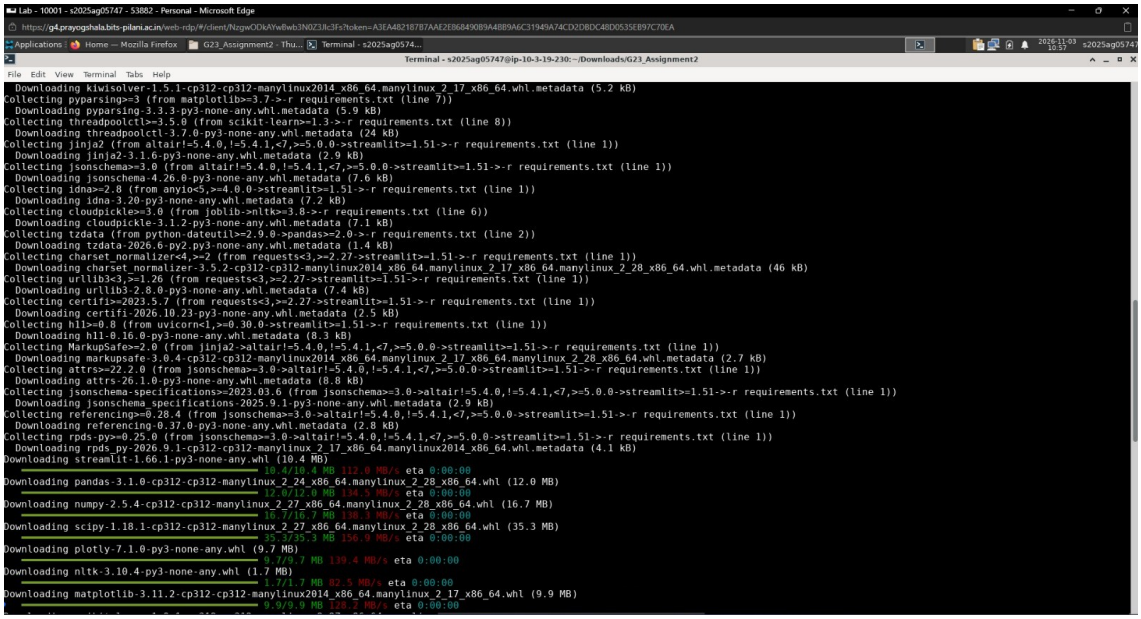


Figure 29: *Virtual Lab terminal: installing the requirements (Streamlit 1.66.1, pandas, numpy, scipy, plotly, NLTK)*



Figure 30: *Virtual Lab terminal: NLTK resources downloaded and the app started with streamlit run*

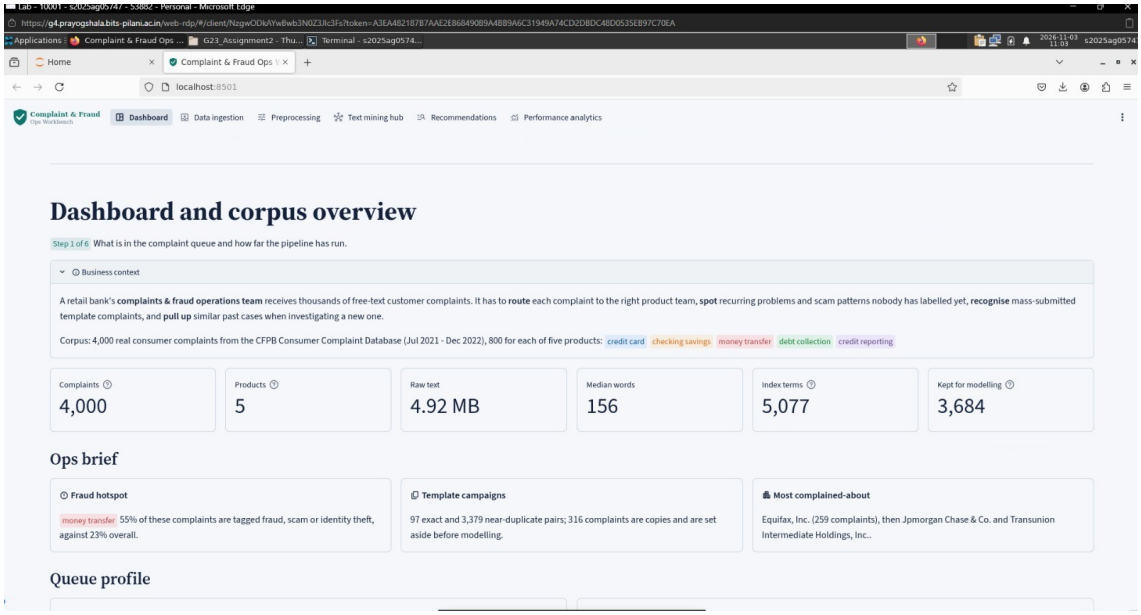


Figure 31: *Virtual Lab browser: Dashboard and corpus overview at localhost:8501, with the Lab session bar and date at the top*

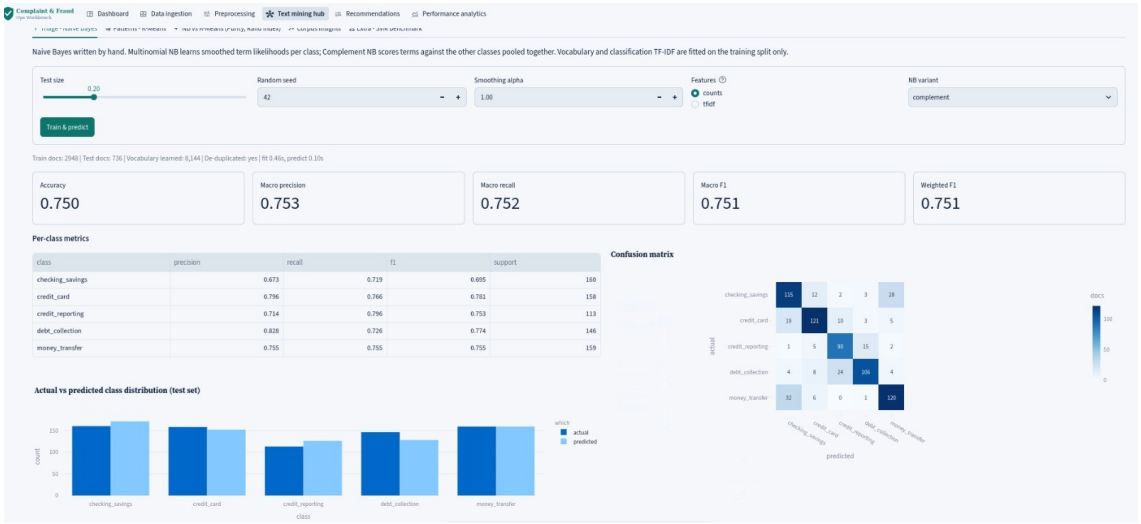


Figure 32: *Virtual Lab: Text mining hub, Naive Bayes tab (accuracy 0.750, macro-F1 0.751, confusion matrix, actual vs predicted)*

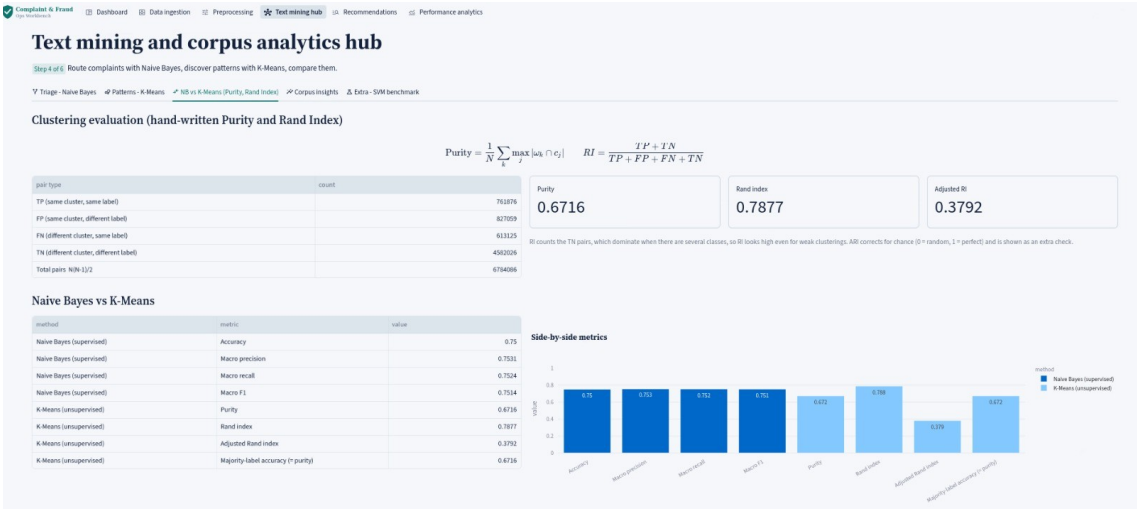


Figure 33: *Virtual Lab: Text mining hub, Purity / Rand Index and Naive Bayes vs K-Means comparison (Purity 0.6716, RI 0.7877, ARI 0.3792)*

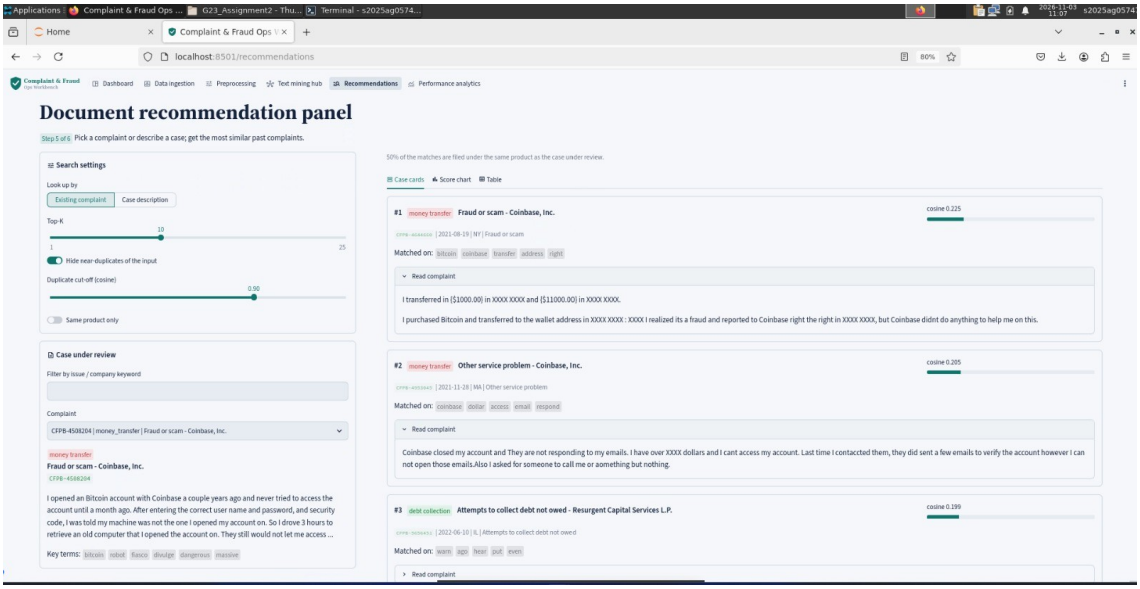


Figure 34: *Virtual Lab: Document recommendation panel with Top-K results, cosine scores and metadata*

Screen recording of the full workflow: virtual_lab/G23_Screen_Recording.mp4 in the submission zip.

Appendix A. Requirement-to-code map

Table 38: *Where each requirement is implemented*

Requirement	Where
A. Streamlit workflow (6 parts)	app.py (top navigation), views/*.py, .streamlit/config.toml (light / dark theme)
B. Ingestion CSV / JSON / folder of text (ZIP)	src/data_io.py: read_table, read_zip_of_txt, split_text_and_meta; views/ingestion.py
B. Corpus statistics, noise, duplicates	src/data_io.py: corpus_stats, noise_profile, find_duplicates, docs_to_drop
B. Dataset preparation	prepare_data.py (sampling from the CFPB parquet shards, text / metadata split)
C. Lowercase, tokenise, stop-words, stem, lemma, n-grams	src/preprocess.py: PrepConfig, preprocess_doc, make_ngrams
C. Inverted index + TF-IDF with length normalisation (manual)	src/tfidf_index.py: InvertedIndex.build, transform
C. Cosine similarity	src/tfidf_index.py: cosine_scores, cosine, similarity_matrix
C. Vocabulary / TF distributions before vs after	views/preprocessing.py: _before_after
D1. Naive Bayes (manual), split, metrics	src/naive_bayes.py (MultinomialNB, ComplementNB); src/pipeline.py: stratified_split, run_naive_bayes; src/metrics.py
D2. K-Means (manual) with init / assign / update / stop log	src/kmeans.py: init_centroids, kmeans_single, kmeans (spherical option)
D3. Purity and Rand Index (manual)	src/metrics.py: purity, pair_counts, rand_index (+ adjusted_rand_index)

Requirement	Where
D4. Visualisations	views/mining.py (confusion matrix, actual vs predicted, scatter, cluster sizes, membership, comparison)
E. Top-K recommendation	src/recommender.py: ContentRecommender; views/recommend.py
Extra: SVM benchmark	src/classifier.py, classifier_experiments.py (scikit-learn LinearSVC on manual TF-IDF)
Report numbers and figures	experiments.py, evaluate_models.py, nb_parameter_search.py, report_extras.py -> results/*.csv, figures/*.png
Tests	tests/test_algorithms.py, tests/test_classifier.py, tests/test_app.py

Appendix B. How to run

```
pip install -r requirements.txt
python -m nltk.downloader stopwords wordnet omw-1.4
python -m streamlit run app.py

# optional: regenerate the tables and figures of this report, and run the tests
python experiments.py
python evaluate_models.py
python classifier_experiments.py
python report_extras.py
python -m unittest discover -s tests
```

The bundled corpus is loaded automatically. Other datasets can be uploaded on the Data ingestion page (data/samples/ has a JSON and a ZIP example). The dark theme is under the app menu (top right) -> Settings -> Theme.

Submission contents (G23_Assignment2.zip): Streamlit source code (app.py, views/, src/, assets/, .streamlit/), the dataset (data/), experiment scripts and saved results (results/, figures/), unit tests (tests/), Virtual Lab screenshots and screen recording (virtual_lab/), this report as PDF, README.md, requirements.txt and G23_Contribution.xlsx.

References

[1] Consumer Finance Monitor (Ballard Spahr), "CFPB ends publication of consumer complaint narratives and data visualizations", 18 Aug 2026. https://www.consumerfinancemonitor.com/2026/08/18/cfpb-ends-publication-of-consumer-complaint-narratives-and-data-visualizations/

[2] Mouwiya, "cfpb-consumer-complaints" (archived copy of the CFPB Consumer Complaint Database), Hugging Face. https://huggingface.co/datasets/Mouwiya/cfpb-consumer-complaints

[3] C. D. Manning, P. Raghavan, H. Schuetze, Introduction to Information Retrieval, Cambridge University Press, 2008 (ch. 6 tf-idf and cosine scoring, ch. 13 Naive Bayes, ch. 16 K-Means, purity and Rand index).

[4] J. D. M. Rennie, L. Shih, J. Teevan, D. R. Karger, "Tackling the poor assumptions of Naive Bayes text classifiers", ICML 2003 (Complement Naive Bayes).

[5] I. S. Dhillon, D. S. Modha, "Concept decompositions for large sparse text data using clustering", Machine Learning 42, 2001 (spherical K-Means).

[6] D. Arthur, S. Vassilvitskii, "k-means++: the advantages of careful seeding", SODA 2007.